

情報科学実験 II

(G-1) シミュレーション (重力による物体の運動)

1 目的

我々の周囲に起こる種々の現象を利用して生活に役立てる為には、着目している現象がどういう時に起こるのか、あるいは、起こせるのかという点を明らかにしなければ達成する事は出来ない。科学はこうした問題に系統的に答えるために人類が営々と築き上げて来たものであり、その成果は現代社会の至る所で援用され豊かな社会が築き上げられている。しかしながら、工業、環境、経済、医学を始めとする様々な分野では対象の高度化とともに、さらに新たな問題も顕在化してきた。そうした問題の多くは現象に対する記述が複雑化し、解析的な現象の予測が困難となっている事に起因している。こうした困難は対象が**大規模で非線形**な特徴を持っている場合に著しい。計算機による数値的な現象の再現による予測(これを**シミュレーション (Simulation)** という)はこうした問題に対し極めて有効な手段を提供する。

本実験では、連続時間系での線形・非線形現象のうち、重力による物体(放物体・天体)の運動の簡単な例について微分方程式、及びこれに対応するブロック線図を用いたモデル化、シミュレーション法を習得する。

2 微分方程式によるモデルの記述とシミュレーションの意義

対象とする現象を記述するためには様々な方法があるが、数式という形で記述することにより、客観性、共有性、一般性が高くなる。数式で記述された対象の事を数理モデルと呼ぶが、ここでは単にモデルと呼ぶ。

数式で現象を記述するためには“対象とする現象”をさらに具体的に述べる必要がある。ここでは、対象は入力(Input)と出力(Output)を持つものとし、それらはなんらかの物理量で測定が可能であるとする。場合によっては入力が無いとされるものもあるが、その場合は0という入力が入っているか、あるいは出力に何ら影響を及ぼさない場合に便宜的に入力が省かれたものである。そうした対象をここでは**システム**と呼ぶ。出力の無いものはシステムではない。システムは図??に示した形式で図示される事が多い。これは後述するブロック線図の最も基本的な物と考えれば良い。

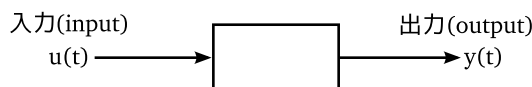


図 1: システム

さてシステムを数式で記述する際、最も多く用いられるのが微分方程式である。時間あるいは空間的に変化する量に着目し、その“増え方”，あるいは“増え方の増え方”がある関係を持つ，といった表現が日常的にも良く用いられる。この表現法を具体的に数式で記述したものが微分方程式そのものである。例えば物体の運動という現象に関し，“速度の増え方（加速度）は加えた力に比例する”という関係を数式で表現すれば

$$\text{力: } F = m \frac{dv}{dt} \quad (\text{ただし } m: \text{ 比例定数 (質量), } v: \text{ 速度}) \quad (2.1)$$

という良く知られた微分方程式(運動方程式)となる。力を入力，物の運動速度を出力と考えればこれはシステムとして捉える事ができ，微分方程式(??)はそのシステムのモデルとなる。ニュートン

やライプニッツに端を発するこうした現象の捉え方は自然科学を推し進めた最も強力な概念であるとする見方もある。

さてシステムがこうした微分方程式によるモデルで記述されたとしても、そのモデルの出力が判らなければシステムの将来の振舞いを予測できないばかりか、観測事実との照合ができず、その妥当性すら検証出来ない。これを行なう為には微分方程式を解く必要がある。解析学はこれらの微分方程式を解く為に発展し、線形の微分方程式に関しては解法が整備されてきた。これによって自動制御理論等が発展した。

しかしながら現在、解析的に解の求められる範囲を逸脱する非線形微分方程式を対象としなければならない問題が各分野で取り上げられるようになった。解析解が求められなくても数値的あるいは近似的な解が求められるならばそれは現実の問題の解としては極めて有効である。モデルの振舞いを求める事を広い意味でのシミュレーションというが、計算機によるシミュレーションが積極的な意味を持つのはこの解析解の得られない場合であると言って良い。

3 ブロック線図によるシミュレーション

3.1 微分方程式とブロック線図

微分方程式で記述されたモデルは、モデルにおける情報の流れや規模の把握、回路としての実現化といった観点から、積分器、算術演算素子などの機能単位をブロックとして表現し、それらを結線した一種の回路図で表される事が多い。この図を**ブロック線図**または**ブロックダイアグラム**と呼ぶ。現在、ブロック線図を描く事でモデルを構成し、シミュレーションの実行が可能なシミュレータが多く普及し、本実験でもその一つを用いてシミュレーションを行なう。

ここでは、簡単な例により、微分方程式からブロック線図を描く手順について説明する。

3.1.1 低次の場合

次のような微分方程式を解いて $y(t)$ を求める事を考える。

$$\frac{dy(t)}{dt} = a \cdot y(t) \quad (3.1)$$

ここで、 t_0 をある定数とし、入力 $x(t)$ を以下のように積分して $y(t)$ を出力するような機能素子を考えよう。

$$y(t) = \int_{t_0}^t x(\tau) d\tau + y(t_0), \quad (t_0 < t) \quad (3.2)$$

容易に判るように、 $y(t)$ の一階微分は積分定数 $y(t_0)$ の値に拘らず

$$\frac{dy(t)}{dt} = \frac{d}{dt} \int_{t_0}^t x(\tau) d\tau = x(t) \quad (3.3)$$

という関係にある。この機能素子は通常下に示したような記号で表される事が多く、この機能素子を**積分器 (Integrator)** と呼ぶ。

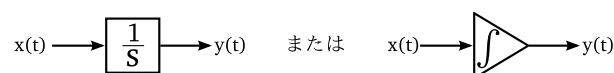


図 2: 積分器

さて、積分器は任意の関数 $x(t)$ を入力する事が出来るが、積分器自身の出力を**接続線** (矢印) を用いて入力に戻す事を考えてみよう。接続線は接続元の変数の値を接続先の変数に代入する機能を有しているものとする¹。この場合、

$$x(t) = y(t) \quad (3.4)$$

$$\text{但し } y(t) = \int_{t_0}^t x(\tau) d\tau + y(t_0), \quad (t_0 < t)$$

となるが、式 (??) を用いて上式の $x(t)$ を消去すると、

$$\frac{dy(t)}{dt} = y(t) \quad (3.5)$$

という微分方程式の関係が得られる。即ち、**積分器の出力を入力に戻す事によって微分方程式を満たすブロック線図を構成出来る事が判る**。そこで、この事を利用して微分方程式 (??) を積分器を用いて表してみよう。式 (??) は、式 (??) の右辺を a 倍する事で得られるから、積分器の出力を a 倍して入力に戻せば式 (??) を満たすように出来る。そのようなブロック線図を描くと下図??のようになる。

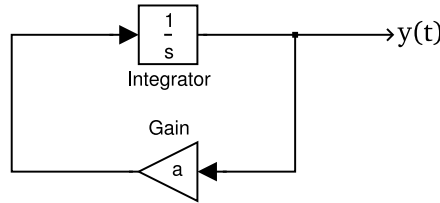


図 3: 微分方程式 (??) を満たすブロック線図

ここで、積分器の下にある三角形のブロックは入力を定数倍して出力する機能素子であり、算術演算ブロックの一つである。これも含めた他の代表的な算術演算ブロックを図??に示す。



図 4: 代表的な算術演算ブロックの例

これらのブロックを用いてさらに複雑な微分方程式をブロック線図で表す事を考えてみよう。次のような 2 次の微分方程式を考える。

$$a \frac{d^2 y(t)}{dt^2} + b \frac{dy(t)}{dt} = y(t) \quad (a \neq 0) \quad (3.6)$$

ここで $x_1(t) = y(t)$, $x_2 = \frac{dy(t)}{dt}$ と変数の置換えを行なうと、式 (??) は

$$\frac{dx_1(t)}{dt} = x_2 \quad (3.7)$$

$$\frac{dx_2(t)}{dt} = \frac{1}{a} x_1 - \frac{b}{a} x_2 \quad (3.8)$$

¹接続線自体は電子回路における導線と同等であるが、ブロック線図におけるブロックでは、電子回路の機能素子の入力インピーダンスが無限大、出力インピーダンスが 0 という理想的な場合に相当する。従って各ブロックの状態が入力端から接続線を経て他へ波及する事は無い。そうすると必然的に接続線には方向が割り当てられるため、矢印で表現される。

と書ける．この場合2つの積分器1, 2を準備し, 各々の出力を $x_1(t)$, $x_2(t)$ とする．まず, 式 (??) から積分器1の入力は積分器2の出力となっているから, 下図のように積分器2の出力を積分器1の入力に接続する．

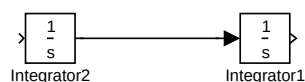


図 5: 2 次微分方程式のブロック線図 1

次に式 (??) から, 積分器2の入力には, 下図のように積分器1の出力の $\frac{1}{a}$ 倍と, 積分器2自身の出力の $\frac{b}{a}$ 倍を加えて接続する．

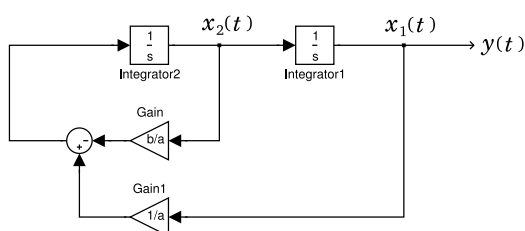


図 6: 2 次微分方程式のブロック線図 2

この例に見るように, 出力同士を接続線で繋ぐ為には何らかの演算ブロック (ここでは加算器) を用いなければならないが, 出力の分岐 (図中黒い点) は接続線の何処でも出来るものとする．

3.1.2 一般の場合 (線形)

式 (??), (??) は行列形式で表せば

$$\frac{d}{dt} \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ \frac{1}{a} & -\frac{b}{a} \end{pmatrix} \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix}, \quad (3.9)$$

$$y(t) = (1 \ 0) \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} \quad (3.10)$$

と書ける．ここで $\frac{d}{dt} \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix}$ は $\begin{pmatrix} \frac{dx_1(t)}{dt} \\ \frac{dx_2(t)}{dt} \end{pmatrix}$ を意味するものとする (以下同様)．システムが入力 $u(t)$ を受ける場合も含むように

$$\frac{d}{dt} \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} u(t) \quad (3.11)$$

$$y(t) = (c_1 \ c_2) \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} + d \cdot u(t) \quad (3.12)$$

と拡張する．式 (??), (??) を合わせたものを (線形) 状態方程式と呼ぶ．また, ベクトル $\begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix}$ を状態変数又は単に状態と呼ぶ．

状態方程式は2次だけでなく、一般の次数にもこのままの形式で拡張でき、ベクトル形式で

$$\frac{d}{dt}\mathbf{x} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad (3.13)$$

$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \quad (3.14)$$

のように記述される。ここで $\mathbf{x}, \mathbf{u}, \mathbf{y}$ は

$$\mathbf{x} = \begin{pmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_m(t) \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_q(t) \end{pmatrix} \quad (3.15)$$

であり、 n, m, q はそれぞれ状態、入力、および出力の次元である²。また $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$ はそれぞれ $n \times n, m \times n, n \times q, m \times q$ の行列である³。

一旦、状態方程式でモデルを記述したならばそこからブロック線図へは次の手順に従えば機械的に変換できる。

手順1 状態方程式の状態の次元 (n) 分だけの積分器を描き、各々の積分器出力に各状態変数 $x_1(t), \dots, x_n(t)$ を割り当てる。

手順2 各積分器の入力は状態の一階微分 $\frac{dx_1(t)}{dt}, \dots, \frac{dx_n(t)}{dt}$ となっているから、状態遷移方程式の右辺を計算し、各状態変数及び入力の結合を算術演算ブロックなどを用いて表し積分器入力側に接続する。これを全ての積分器について行なう。

手順3 出力は状態変数、及び入力の結合であるから、それらを手順2と同様にブロックを組み合わせ構成する。

この手順を用いて、式(??)、式(??)で表される状態方程式に対応するブロック線図を描くと図??のようになる。

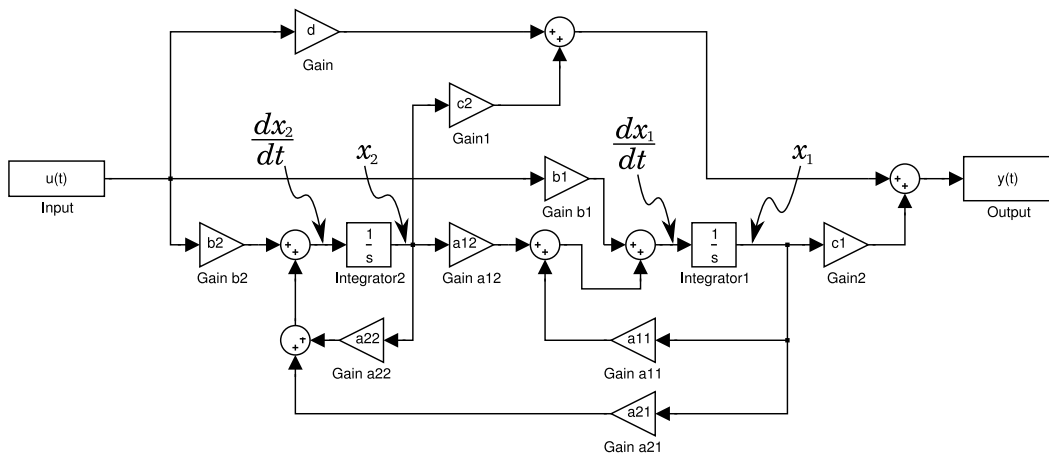


図 7: 2 次微分方程式のブロック線図

²ベクトル表記は $\vec{x}$ と書く事もあるが、多くの場合太字で $\mathbf{x}$ と書かれる。

³書籍によってはフィードバックの観点から $\mathbf{D}\mathbf{u}$ の項が無い物もある。

3.1.3 一般の場合（非線形）

例えば状態変数間での積や $\sin()$, $\exp()$ などの関数が現われる場合には，微分方程式であっても式 (??),(??) のように書く事ができなくなる．こうした場合（非線形）状態方程式は，

$$\frac{d}{dt} \begin{pmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{pmatrix} = \begin{pmatrix} F_1(x_1(t), x_2(t), \dots, x_n(t), u(t)) \\ F_2(x_1(t), x_2(t), \dots, x_n(t), u(t)) \\ \vdots \\ F_n(x_1(t), x_2(t), \dots, x_n(t), u(t)) \end{pmatrix} \quad (3.16)$$

$$y(t) = G(x_1(t), x_2(t), \dots, x_n(t), u(t)) \quad (3.17)$$

と表される．通常，右辺の $F_1, \dots, F_n, G$ は行列形式でなくても， $x_1(t), x_2(t), \dots, x_n(t), u(t)$ の関数として具体的に記述されているから，シミュレータに準備されている種々のブロックを用いてそれらの関数を構成する事が出来る．従って，前節で述べた手順の 2 と 3 の変数の結合に定数倍，加算器のみでなく他のブロックも用いればブロック線図を構成できる．

例えば状態方程式

$$\frac{d}{dt} \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} = \begin{pmatrix} ax_1(t)u(t) \\ bx_1(t)x_2(t) + u(t) \end{pmatrix} \quad (3.18)$$

$$y(t) = x_1(t) \quad (3.19)$$

は式 (??),(??) の形に変形できないが，先の手順に従えば図??のようにブロック線図として表す事が出来る．

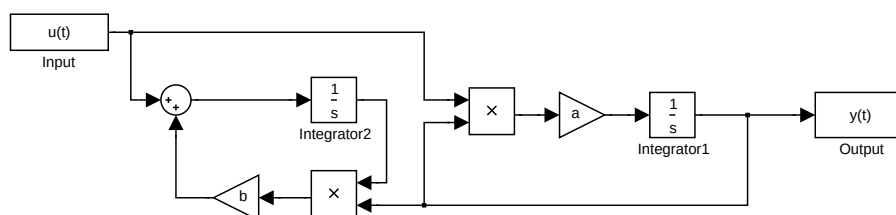


図 8: 非線形 2 次微分方程式のブロック線図

3.2 初期状態の設定

ブロック線図などを用いてモデルを記述しても，各状態変数の初期値を設定しなければシミュレーションを行なう事はできない．積分器を用いるブロック線図ではこの初期値は各積分器の積分定数に相当する．

3.3 微分方程式の数値解法の選択

微分方程式は数学で学習したように解析的に解ける物もあるが，数値解析で学習したように数値解法を用いなければならない物が大半である．数値解法の内，4 次の Runge-kutta 法は簡単で汎用性も高く良く用いられている．多くのシミュレータでは，さらに，高次の方法や，計算ステップ幅を自動的に変化させる事で精度の高い解を効率良く得るための方法をサポートしている．特に Runge-Kutta 法の 4 次と 5 次の公式で得られる解の差がある基準（相対誤差という）以下になるようにステップ幅を小さくしていく方法は極めて汎用的である．

ブロック線図を構成したら，この数値解法の中から適当な物を選ぶ必要がある．

3.4 シミュレーションの実行及び結果の評価

積分器初期値を設定し、適当な数値解法を選択したら、シミュレーションを実行できる。通常は、結果をグラフ表示し、その妥当性について評価を行なう。シミュレーション結果の評価は一般的には難しいが、本実験では後で述べる具体例について段階的に評価を行なう。

4 放物体

重力が働いている場所で投げ上げられた物体の運動を放物運動と呼ぶ。ここでは空気抵抗が無く簡単な場合、空気抵抗のある場合、人工衛星、多体問題についてシミュレーションを行う。

4.1 実験 1: 空気抵抗の無い場合

地表を直線 (x 軸) とし、これを含む鉛直面内 (鉛直方向を y 軸) で質量 $m(\text{kg})$ の物体を水平初速度 $v_x(0)$ 、垂直初速度 $v_y(0)$ で放った。図??に示したようにこの物体には常に鉛直下方に重力 mg (但し g は重力 (万有引力) 定数 (9.8m/sec^2) が働いている。

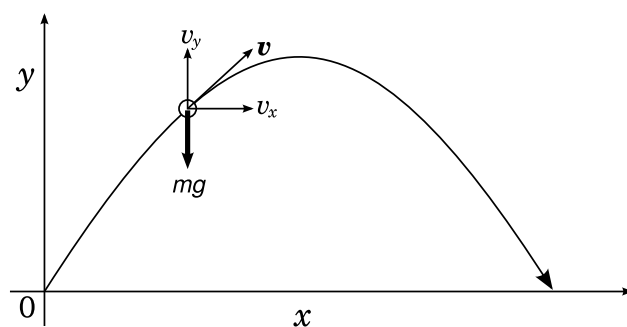


図 9: 放物体の運動 (空気抵抗の無い場合)

この物理現象は入力を重力 mg (定数関数と見る)、出力に物体の位置 $\mathbf{p} = (x(t), y(t))^T$ ($(\cdot)^T$ は転置を表す) を取る事でシステムと見做す事が出来る。

さて、運動方程式は、式 (??) で与えられるが、力 $\mathbf{F}(t)$ や速度 $\mathbf{v}(t)$ は、大きさと方向を持つベクトルであるから、2次元平面内での方程式

$$\mathbf{F}(t) = m \frac{d\mathbf{v}(t)}{dt} \quad (4.1)$$

は行列形式で

$$\begin{pmatrix} F_x(t) \\ F_y(t) \end{pmatrix} = m \frac{d}{dt} \begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix} \quad (4.2)$$

と書ける。今、図??の物体には鉛直下方に重力 ($-mg$) のみが働くから、方程式

$$\begin{pmatrix} F_x(t) \\ F_y(t) \end{pmatrix} = m \frac{d}{dt} \begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix} = \begin{pmatrix} 0 \\ -mg \end{pmatrix} \quad (4.3)$$

が得られる。また、 $v_x(t), v_y(t)$ はそれぞれ $x(t), y(t)$ の時間微分で与えられるので、

$$\begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix} = \frac{d}{dt} \begin{pmatrix} x(t) \\ y(t) \end{pmatrix} \quad (4.4)$$

であり、式 (??) 式 (??) を同時に行列形式で表現することで状態方程式が以下のように導かれる。

$$\frac{d}{dt} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{-1}{m} \end{pmatrix} mg \quad (4.5)$$

$$\mathbf{p}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} \quad (4.6)$$

上式は式 (??), (??) と同じ形式であるから線形状態方程式である。

4.1.1 課題 1-1

??で述べた手順に従ってブロック線図を作成せよ。付録の手順に従ってブロック線図をレポートに貼り込むこと。あまりに小さく印刷しないよう注意すること。

4.1.2 課題 1-2

初速度を $v_x(0) = 4.0m/sec$, $v_y(0) = 12.0m/sec$ とし, (x, y) の原点から物体を放出した後 2.5 秒間をシミュレーションし, 物体の位置を (x, y) 上のグラフとせよ。ディスプレイに表示されたグラフウィンドウをキャプチャ(ウインドウマネージャに付属しているソフトを利用する)して eps ファイルに変換するなどしてレポートに貼り込めば良い。以降の課題でも同様であるが, MatLab の Workspace にデータを引き渡すブロック (ToWorkspace ブロック: 参考 2 を参照のこと) があるのでそれを利用すれば、品質の高いグラフが得られる。

4.1.3 課題 1-3

放つ際の速度 ($\sqrt{v_x^2(0) + v_y^2(0)}$) を一定 ($= 10m/sec$) とし, 角度を地表に対して上方 0 度から 90 度まで変化させた際の到達距離 (x 軸と再度交わる x 座標) をグラフにせよ。何度の時に最も遠くへ到達するか。この角度を精密に計測するにはどうしたら良いか, 各自考察し, 出来るなら現在の環境で最高の精度で求めてみよ。

4.1.4 課題 1-4

解析的に到達距離を求めシミュレーション結果と比較し, 誤差が生じたならば何故かを考察せよ。またこの誤差を小さくするにはどうしたら良いかを考察せよ。

4.2 実験 2: 空気抵抗のある場合

実験 1 では空気抵抗を考慮しなかったが, 空気中を物体が進行する場合, 速度の大きさの二乗⁴に比例した抵抗力を, 進む方向とは逆向きに受ける。

⁴速度が小さい場合は速度に比例するが, 速度が大きい場合は二乗に比例する。

ある時刻 t での速度ベクトルを $\mathbf{v}(t)$ とすると、その逆向きの単位ベクトルは $-\frac{\mathbf{v}(t)}{|\mathbf{v}(t)|}$ であるから、これに速度の大きさの 2 乗: $|\mathbf{v}(t)|^2$ と比例係数 α を掛けると、空気抵抗は $-\alpha \mathbf{v}(t) |\mathbf{v}(t)|$ となる。今速度ベクトルは $\mathbf{v}(t) = (v_x(t), v_y(t))^T$ であったから、x 軸方向 y 軸方向それぞれに加わる空気抵抗力は

$$\begin{pmatrix} -\alpha v_x(t) \sqrt{v_x^2(t) + v_y^2(t)} \\ -\alpha v_y(t) \sqrt{v_x^2(t) + v_y^2(t)} \end{pmatrix} \quad (4.7)$$

となる。従って位置、速度、加速度は以下のような方程式に従う。

$$\frac{d}{dt} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} = \begin{pmatrix} v_x(t) \\ -\alpha v_x(t) \sqrt{v_x^2(t) + v_y^2(t)} \\ v_y(t) \\ -g - \alpha v_y(t) \sqrt{v_x^2(t) + v_y^2(t)} \end{pmatrix} \quad (4.8)$$

上式は式 (??), (??) と同じ形式に直す事が出来ない所以非線形状態方程式である。

4.2.1 課題 2-1

??で述べた手順に従ってブロック線図を作成せよ。

4.2.2 課題 2-2

放つ際の速度 ($|\mathbf{v}| = \sqrt{v_x^2(0) + v_y^2(0)}$) を一定とし、角度を地表に対して上方 1 度から 90 度まで変化させた際の到達距離 (x 軸と再度交わる x 座標) をグラフにせよ。何度の時に最も遠くへ到達するか。但し α , 初速度: $|\mathbf{v}|$ の値は実験当日指示する。

5 人工衛星

5.1 実験 3: 衛星が一つの場合

放物体の初速度が小さい場合は実験 1, 2 でのように地表は無限長の直線 (平面) であると仮定できた。しかし、人工衛星のような放物体や天体の運動では地球及び放物体を質点とみなした解析が必要である。

今、図??に示したように (x, y) 平面内原点に質量 M の物体:A (地球などの天体と考える) があって動かないものとし、これに対して自由に動ける質量 m の物体:B(人工衛星) があるものとする。

B の位置を $\mathbf{p}(t) = (x(t), y(t))^T$ とし、これに働く力: $\mathbf{F}(t) = (F_x(t), F_y(t))^T$ は、“距離の逆二乗則”から

$$\mathbf{F}(t) = -\frac{gMm}{|\mathbf{p}(t)|^2} \frac{\mathbf{p}(t)}{|\mathbf{p}(t)|} \quad (5.1)$$

$$\text{但し } |\mathbf{p}(t)| = \sqrt{x^2(t) + y^2(t)} \quad (5.2)$$

である。従って、運動方程式を成分毎にベクトルで表せば

$$\begin{pmatrix} F_x(t) \\ F_y(t) \end{pmatrix} = m \frac{d}{dt} \begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix} = mgM \begin{pmatrix} -\frac{1}{(\sqrt{x^2(t) + y^2(t)})^3} x(t) \\ -\frac{1}{(\sqrt{x^2(t) + y^2(t)})^3} y(t) \end{pmatrix} \quad (5.3)$$

と書ける．簡単のためにここでは， $gM = 1$ とし，式 (??) も同時に方程式に組み込めば，B に関する状態方程式

$$\frac{d}{dt} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} = \begin{pmatrix} v_x(t) \\ -\frac{1}{\sqrt{x^2(t) + y^2(t)}^3} x(t) \\ v_y(t) \\ -\frac{1}{\sqrt{x^2(t) + y^2(t)}^3} y(t) \end{pmatrix} \quad (5.4)$$

となる．

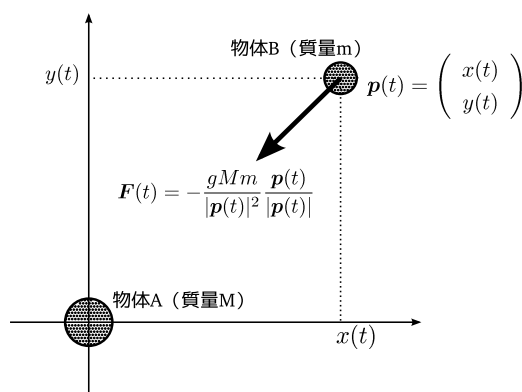


図 10: 物体に働く力

5.1.1 課題 3-1

??で述べた手順に従ってブロック線図を作成せよ．

5.1.2 課題 3-2

B の初期位置を， $(x, y) = (0.0, 20.0)$ ，初期速度を $(v_x, v_y) = (0.15, 0.0)$ とし，B の軌道を描け．

5.1.3 課題 3-3

適当に初期値を変化させ，シミュレーションを行ってみよ．特に第一宇宙速度，第二宇宙速度について (Wiki など) で調べ，第二宇宙速度以下ならば必ず元の位置を通る軌道を描く事を確認せよ．($gM = 1$ とし，半径 (R) は人工衛星の初期位置の物体 A からの距離とすれば良い)

5.1.4 課題 3-4

ケプラーの第二法則について調べ，面積速度を出力するような機能をブロック線図に付加せよ．その出力が時間的に変化しない事を示すグラフ表示を行い，ケプラーの第二法則が成立している事を示せ．

5.2 実験 4: 二つの惑星間の衛星の運動

2つの物体AとBの間を飛行する物体Cの運動を考える．図??に示したように，CにはA及びBからの引力の合力が働く．

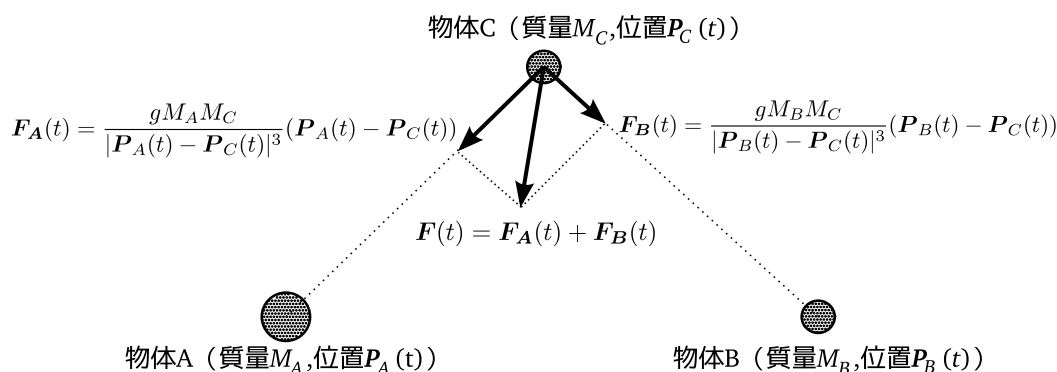


図 11: 2つの物体からの引力

A,B,C 各々の位置を $\mathbf{P}_A(t), \mathbf{P}_B(t), \mathbf{P}_C(t)$, 質量を M_A, M_B, M_C とすると, Cに働く合力 $\mathbf{F}(t)$ は

$$\mathbf{F}(t) = \frac{gM_AM_C}{|\mathbf{P}_A(t) - \mathbf{P}_C(t)|^3}(\mathbf{P}_A(t) - \mathbf{P}_C(t)) + \frac{gM_BM_C}{|\mathbf{P}_B(t) - \mathbf{P}_C(t)|^3}(\mathbf{P}_B(t) - \mathbf{P}_C(t)) \quad (5.5)$$

となる．ここで実験3と同様，簡単のために，A及びBは各々(0,0), (1,0)にあって動かないものとし，Cの位置を $(x(t), y(t))$ とする．また， $M_A = 1, M_B = \lambda, g = 1$ として上式を成分毎に書けば

$$\begin{cases} F_x(t) = M_C \left(-\frac{1}{r_a^3(t)}x(t) + \frac{\lambda}{r_b^3(t)}(1-x(t)) \right) \\ F_y(t) = M_C \left(-\frac{1}{r_a^3(t)}y(t) - \frac{\lambda}{r_b^3(t)}y(t) \right) \end{cases} \quad (5.6)$$

$$\text{ただし} \begin{cases} r_a(t) = |\mathbf{P}_A(t) - \mathbf{P}_C(t)| = \sqrt{x^2(t) + y^2(t)} \\ r_b(t) = |\mathbf{P}_B(t) - \mathbf{P}_C(t)| = \sqrt{(1-x(t))^2 + y^2(t)} \end{cases} \quad (5.7)$$

となる．これを用いれば状態（運動）方程式は以下ようになる．

$$\frac{d}{dt} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} = \begin{pmatrix} v_x(t) \\ -\frac{1}{r_a^3(t)}x(t) + \frac{\lambda}{r_b^3(t)}(1-x(t)) \\ v_y(t) \\ -\frac{1}{r_a^3(t)}y(t) - \frac{\lambda}{r_b^3(t)}y(t) \end{pmatrix} \quad (5.8)$$

5.2.1 課題 4-1

??で述べた手順に従ってブロック線図を作成せよ．

5.2.2 課題 4-2

物体Aは半径0.2の円形をしているとする．この円周上の何処かからCを放し再び円周上の何処かに戻って来るような軌道の内，Bがこの軌道の内側となるような初期条件を二つ以上見付けCの軌道を図示せよ．但し $\lambda = 0.4$ とする．また，この軌道がAとBの中間で交差（8の字状）するような初期条件を一つ以上見付け，同様に結果を図示せよ．

5.3 実験 5: 多体問題（二つの衛星の場合）

5.3.1 課題 5-1

実験 4 で物体 B も自由に動ける場合を考える．状態方程式を導け．但し $G = 1$, $M_A = 1$ とせよ．

5.3.2 課題 5-2

導いた状態方程式からブロック線図を描け．

5.3.3 課題 5-3

$M_B = 0.4$, $M_C = 0.2$, $(x_b(0), y_b(0)) = (1, 0)$, $(v_{x_b}(0), v_{y_b}(0)) = (0, 1)$, $(x_c(0), y_c(0)) = (0.141421356, 0.141421356)$ (2.5, 1) として 40sec 間の B と C の運動を XY 平面上に図示せよ．但し可変ステップアルゴリズムで相対精度を $2.8\text{e-}14$ 程度とせよ．

また, $v_{x_b}(0) = 0$ であった所を $v_{x_b}(0) = 0.00001$ とし, 後は全く変えないでシミュレーションを行ないこの微小な変化が軌道に及ぼす影響に付いて考察せよ．

6 自由課題 1: 自然現象

力学現象, 電気回路, 生命現象などの中から好きな現象のモデルを構成し, シミュレーションを行ないモデルの妥当性を検証せよ．

7 自由課題 2: 社会・経済現象

社会経済現象の中から好きなものを選んでモデルを構成し, シミュレーションを行ない, モデルの妥当性を検証せよ．

8 レポート作成上の注意

1 表紙

表紙は任意の形式で良いが次の内容が記載されている事。

(あ) 実験のタイトル：

情報科学実験 II (G-1) シミュレーション

第 1 回目レポート 『放物体 I』

第 2 回目レポート 『放物体 II 及び人工衛星 I』

第 3 回目レポート 『人工衛星 II 及び多体問題』

(い) 学籍番号, 名前

(う) 実験日

2 目的

各項目毎に実験の目的を書く事．しかし、音声のシミュレーションの場合とは異なり“実験による仮説の検証”を目指してはいない．“放物体”をシミュレーションし, 実際に物を投げる実験を行う訳ではないので, 実験による仮説の検証まで行わない．あくまで“理論値を計算する(予測する)”所までであるので, 科学の本来の意味の“実験”は本テーマに含まれていない点に注意して欲しい．

3 理論

放物体、人工衛星に関しては指導書にも書いたように物理学から、微分方程式で記述されたモデル(理論、仮説)が与えられているので、簡単でも良いから、項目毎にそれを書く事。課題の中で、解析解とシミュレーション結果を比較する物があるので、この理論の中で、数値を代入する前の解析解も導出しておくのが良い。レポートは3回であるが、項目は3個以上あるので、各項目毎に 理論→結果→評価という順で作成すること。

現象の理論とは別に、本テーマでは、シミュレーション自体の理論がある、その理論は“用いる数値解法が正しい結果を与える根拠”である。従って解析解とシミュレーション結果がどのように異なるかをその理論で説明出来る筈であるから、その説明に必要な理論(数値計算法に関する)を述べるべきである。しかし、全員がそれに関して講義を受けている訳ではないので、これに関しては出来る範囲で良い。

4 実験結果

実験結果はグラフが大半であるので、何のどういう結果をどのようにグラフにしたのかを図番号を参照しながら必ず文章で説明する事。

5 実験結果に対する評価・考察

放物体に関しては、日常経験している事を思い出せば、ある程度結果の評価は可能であるが、定量的な評価は出来ない。従って本テーマにおいて評価とは“解析解との比較”のみとなる。解析解が求まる場合はその旨課題に示してあるのでその場合に限り、理論の項目で書いた事と呼応させて(数値を代入する)記述せよ。多体問題になると解析解は無いので、シミュレーション結果を評価出来ない。さらに実際に衛星を飛ばす事も出来ないので検証出来ない。従って、定量的な評価は本テーマの中だけでは出来ない。その場合には、ロケットの軌道が何故あのようになっているのか等という点から定性的な考察を行えば良い。また、科学の中におけるシミュレーションの位置付け、意義に関して自由に考察を行っても良い。書籍を漁れば色々な蘊蓄が述べられている(特にカオス、複雑系など)ので、自分なりに理解した点をまとめれば良い。

6 実験に対する改良点などあれば何故そのように思うかという根拠を示した上で述べて欲しい。感想などを書いても良い。

7 参考文献、参照ホームページは文章中で番号を付け、レポートの最後に文献の表を付けよ。

8 レポートは必ず次週の実験中、最後の実験に関しては次週の木曜日までに提出する事。返却され、訂正・加筆の必要な場合、再提出の期限はその都度言う(原則1週間)。期限を過ぎて提出されるレポートに関しては10%/日の割合で減点とする。

参考文献

- [1] 垣田高夫, 大町比佐栄 訳: “微分方程式で数学モデルを作ろう”, 日本評論社 (1990).
- [2] “Using Simulink version2.1”, The Math Works Inc.(1997).

付録：Simulink によるモデルの記述・シミュレーション

本実験では MATLAB 上のモデリング・シミュレーション用ツールキット:Simulink を用いてシミュレーションを行なうが、ここでは、その簡単な使用法を説明する。詳しい事はマニュアル [?] を参照の事。

1 ブロック線図の作成

1.1 Simulink の起動

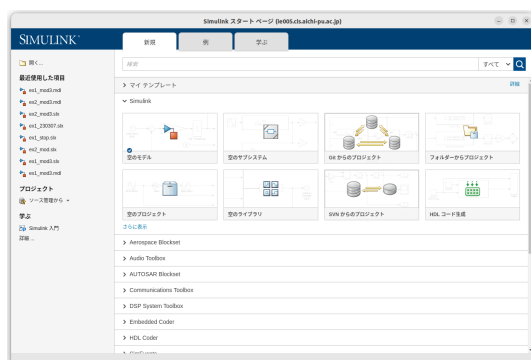
Matlab(2024 年度では 2023b) をまず起動する。Linux 環境下では

```
% matlab ↓
```

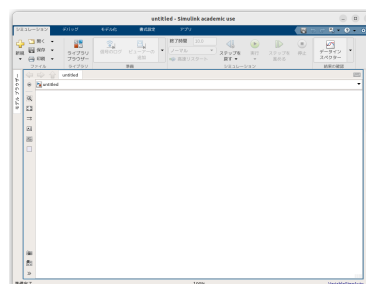
と入力する。そして、そのプロンプト (>>) 上

```
>> simulink ↓
```

と入力、または、Matlab ウィンドウの上部メニューバーの中にある“Simulink”をクリックすると、図??(a)のような新しいウィンドウ（Simulink スタートページ）が開かれる。



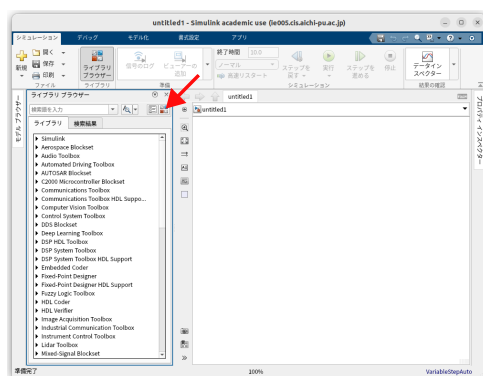
(a) Simulink スタートページウィンドウ



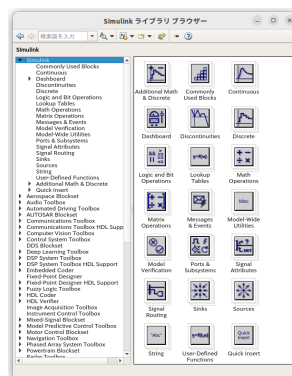
(b) モデルウィンドウ

図 12: Simulink スタートページウィンドウとモデルウィンドウ

Simulink スタートページウィンドウの中の“v Simulink”と銘打たれたサブウィンドウの中の左上に位置する“空のモデル”と書かれたバナーをクリックすると、図??(b)のような何も表示されていないモデルウィンドウが開かれ、スタートページウィンドウは閉じられる。この操作で新しいモデルを記述する準備ができる。



(a) ライブラリブラウザ(その1)



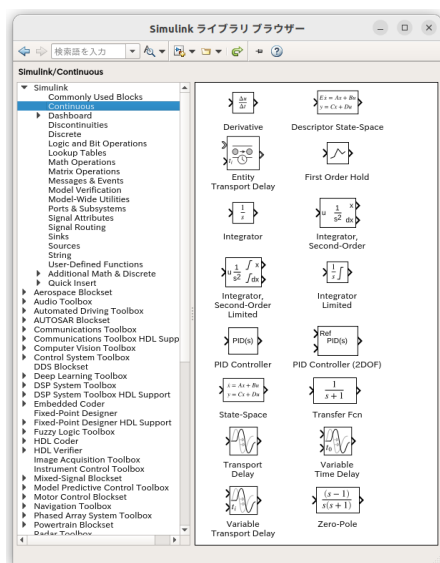
(b) ライブラリブラウザ(その2)

図 13: モデルウィンドウとライブラリブラウザーの起動

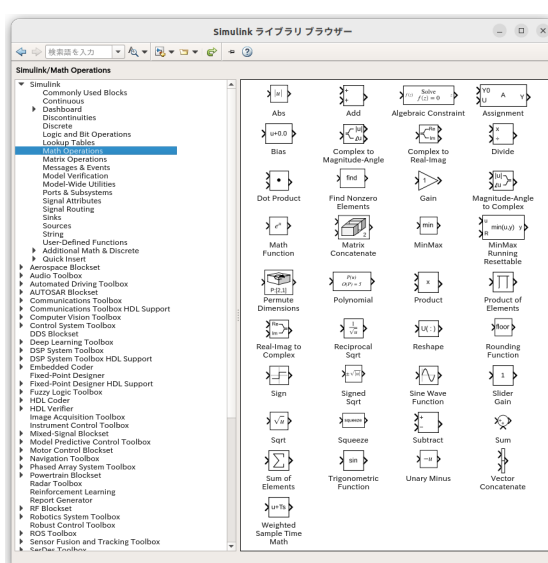
1.2 モデルの記述

1.2.1 ブロックライブラリ

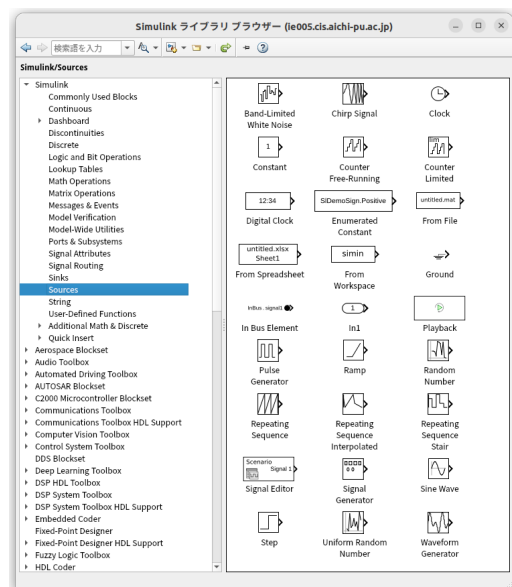
モデルウインドウの上部には“シミュレーション”、“デバッグ”、“モデル化”などのタブが並んでおり、“シミュレーション”タブが選択された状態となっている。この状態で、タブの列の下に、“ライブラリブラウザ”というタブがあるのでそれをクリックすると、モデルウインドウの左部分に?? (a) のようにライブラリブラウザが開かれる。また、現れたライブラリブラウザの右上のアイコン（赤の矢印で示された部分）をクリックすると、?? (b) に示したような新たなブラウザウインドウが開かれる。ブロックライブラリブラウザには“Sources”、“Sinks”、“Continuous”といったアイコンが並んでいるがそれぞれはブロックの性質毎の分類である。各分類の中には Simulink で提供されるブロックが納められている。アイコンをダブルクリック（または左のリストをクリック）することで図??に示したような新たなウインドウが現れ、その中に種々のブロックがあることがわかる。積分器は“Continuous”に納められている。算術演算ブロックは実験指導書本文で述べた通りである。尚、モデルウインドウの左部分に現れたライブラリブラウザからでも、同様にブロックを選択できるようになっている。ライブラリブラウザは×印をクリックすれば閉じることができる。



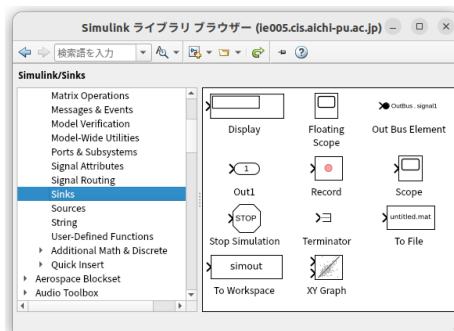
Continuous (連続系)



Math Operators (算術演算)



Sources (信号源)



Sinks (信号端末)

図 14: ブロックライブラリの内容の例

1.2.2 ブロックの選択

例えば状態方程式

$$\frac{d}{dt} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{-1}{m} \end{pmatrix} mg \quad (1.1)$$

$$\mathbf{p}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} \quad (1.2)$$

が与えられ、そのブロック線図を描く事を考える。上式は状態方程式の形をしているが、右辺を展開し、 x, y について独立に2次の微分方程式を立てると

$$\begin{cases} \frac{dx(t)}{dt} = v_x(t) \\ \frac{dv_x(t)}{dt} = 0 \end{cases} \quad \begin{cases} \frac{dy(t)}{dt} = v_y(t) \\ \frac{dv_y(t)}{dt} = -g \end{cases} \quad (1.3)$$

となる。状態変数は4個あるので、手順に従えば積分器は4個必要である。しかし、 $v_x(t)$ を出力とする積分器の入力 ($\frac{dv_x(t)}{dt}$) には常に0が入力されているため、 $v_x(t)$ は常にこの積分器の積分定数 (この場合は水平方向初速度 $v_x(0)$) となる。従ってここには積分器を用いなくても定数を出力するブロックを用いる事と等価になる。ここではまず、ブロックライブラリの“Sources”の中から“Constant”ブロックを選ぶ。ブロックライブラリの“Sources”アイコンをダブルクリックすると、図??の中の“Sources”ウインドウに切り替わるから、その中の“Constant”ブロックをドラッグしながらブロックをモデルウインドウ内の適当な位置に移動させボタンを離す(ドロップ)と、その位置に“Constant”ブロックが描かれる(下图)。この時、ブロックを置けない位置には、進入禁止の道路標識に似たシンボルになっている。

ブロックは幾つかの設定が必要なパラメータを持っている。Constantブロックは出力する定数値を設定しておかねばならない。Constantブロックは最初の状態では内部に“1”と書かれているが、これが定数値を表している。

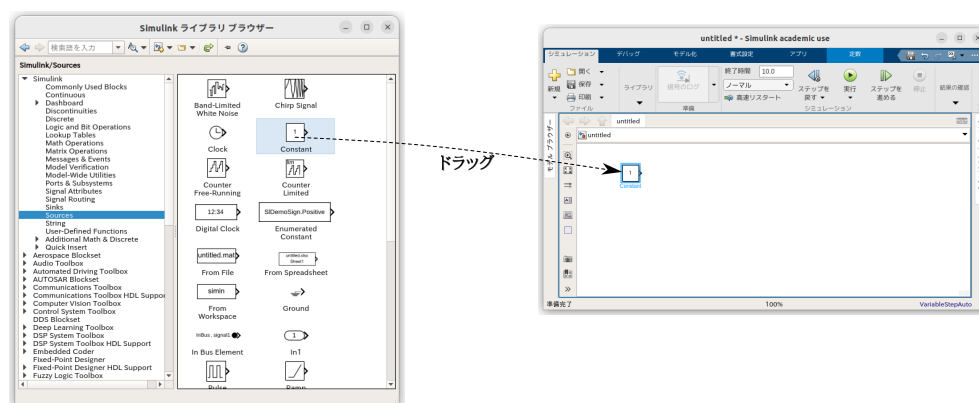


図 15: “Constant” ブロックのモデルウインドウ内へのドラッグ&ドロップ

ブロック周囲が水色に縁取られており、マウスカursorをブロック上に置くと四隅に正方形が現れるが、これはこのブロックが**選択**されている事を示すものであり、この状態でブロック内の数値“1”の上にマウスカursorを重ねるとカーソル形状が“T”に変化し、別の数値を入れられる状態となる。選択されているブロックに対しては、モデルウインドウの上部タブの“書式設定”の中の回転(時計回りに90度回転)などの処理が有効となる。これらの処理に関しては次節で説明する。

1.2.3 ブロックのコピー・移動・回転・削除・名前の変更

次にモデルライブラリ“Continuous”の中から積分器 (Integrator) を同様にドラッグし、モデルウインドウ内に移動させる。積分器はこの例の場合 3 個必要となるから、3 回モデルライブラリウインドウからドラッグしても良いし、1 個の積分器をモデルウインドウ内でコピーしても良い。コピーの方法は何種類もある。最も簡単なのは Ctrl キーを押しながら、図??に示したようにブロックをコピー先までドラッグすれば、“Integrator1”という名前のついたブロックが新たに現れる。これ以外にも、ブロック内で右クリックしそのメニュー内のコピーと貼り付けの機能を使う方法、モデルウインドウ上部の“編集”メニューからコピーを選ぶ方法もある。また、ブロックを選択しておき、キーボードの Ctrl キーを押しながら c キーを押せば (Ctrl+c と略記) システム内のバッファ領域にストアされ、Ctrl+v で同様にコピーを行う事が出来る。覚える事が出来ればこうしたキー割り付けによる操作の方が効率的である。キー割り付けは各サブメニューに表示されている

また、削除はブロックの右クリックによるメニューの中から削除を選ぶか、ブロックを一回クリックして選択しておき、Ctrl+x で削除できる。

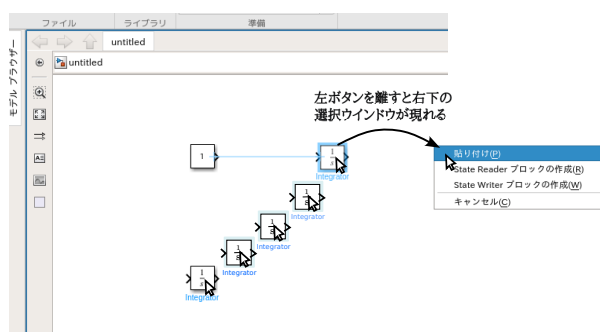


図 16: ブロックのコピー

選択されたブロックの移動はマウスのドラッグでもカーソルキーでも行なえる。また図??(a)に示したように、ブロック内を左クリックすると、ブロックの上に (...) が表示されこれをクリックすれば、いくつかのシンボルが現れる。このシンボルの左から 2 つ目が、“ブロック名を表示”となっているので、これを選べばブロック名 (Constant) が黒色に変わり、常に表示されるようになる。また、ブロックは左が入力で右が出力である物が多いが、同図 (b) に示したようにブロック内を右クリックする事で現れるメニュー中の“書式設定”を選べばさらにサブメニューが現れ“時計方向に 90 度回転”を選べば上下左右に出力方向を変化させる事ができる。さらに、“ブロックパラメータ”を選ぶと、そのブロックに特有の値、Constant ブロックにおいては、出力する定数の値を設定する事ができる。ブロックパラメータについては後述する。



図 17: ブロック内でのクリック

式 (??),(??) では出力は状態変数 $x(t)$ と $y(t)$ の 2 次元であるから、ここでは、モデルライブラリ:“Sinks”の中の“XY Graph”ブロックを用いる。また、式 (??) の y に関する微分方程式では、 $v_y(t)$ を出力する積分器の入力に g という定数が入力されているので、“Constant”ブロックをもう一つモデルウインドウ内に置く。

さて、上にも述べたが、コピーされた新しいブロックでは名前 (ブロックの下に表示されている英文字) に元のブロックの名前の後に “1”, “2” という番号が付されて表示される。これは、モデルウインドウ内に同種のブロックが存在する時、モデルライブラリからドラッグしても同様である。これはブロックの識別のためにシステムが自動的に付したものであるが、気に入らない場合は変更できる。

ここでは後の説明の為に、“Constant” および、“Integrator” と表示されているブロック (最初にモデルライブラリからドラッグしてきた物) の名前を “Constant0”, “Integrator0” と変更しよう。マウスカーソルを名前の文字の上の何処かに置きクリックすると、マウスクリックされた文字の間に入力の為のカーソル (I) が現われる。表示の文字が小さくて見えない場合は Ctrl++ (あるいはマウスのホイールの操作) で拡大表示できる。

キーボードのカーソルキー (→) を押して最後の文字の後までカーソルキーを移動させ、“0” をキーボードから入力する。0 が入力された事を確かめ、マウスカーソルを別の場所に移動させクリックすれば名前は確定する。同様にして積分器の名前も変更する。ブロックの名前を変更する際、ブロックウインドウ中に同じ名前のブロックが存在すれば、システムは同一の名前がある事を警告する。ブロック名は一つのモデル中、唯一でなければならない。

1.2.4 ブロックの接続

さてこれらを接続線により接続する。各ブロックには左右にマーク “>” が付されている。ブロック側から出る方向に開いているマークが入力ポート (端) であり、出る方向に閉じているマークが出力ポートである。出力ポートにマウスカーソルを近づけると、マウスカーソルが矢印からクロスヘアカーソル (“+” 印) に変化する。最初に置いた “Constant” ブロックの出力ポートにマウスカーソルを近づけクロスヘアカーソルにしたところでマウスの左ボタンを押してマウスを少し移動させると、右あるいは左に水平の破線 (赤) が引かれる。(中ボタンでドラッグさせると斜めの線を引く事ができる。) このまま積分器の入力ポート近くまでドラッグしていき、そこでボタンを離すと両ブロック間に実線矢印の接続線が現われる (図??)。

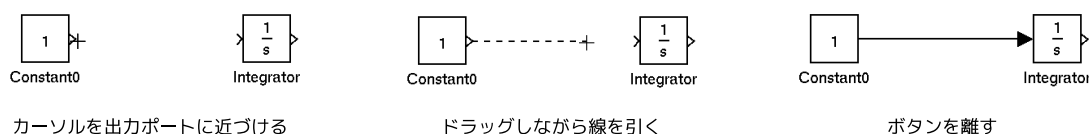


図 18: ブロックの接続

また、ブロックを選択しておき、Ctrl キーを押しながら接続先のブロックを左クリックすると、自動的に接続される。このようにして、3つの積分器、2つの定数、1つの XY グラフブロックを式 (??) に従って接続したものが図??である。

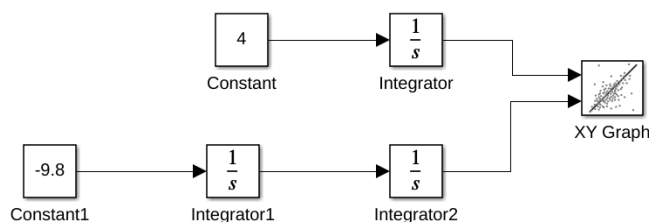


図 19: 接続されたブロック

1.2.5 ブロックパラメータの設定

モデルの記述はブロック間の接続と、積分器初期値等のブロックパラメータの設定によって成り立っている。図??の状態では未だパラメータの設定が出来ていない。パラメータを設定するためには、設定したいブロック上にマウスカーソルを移動させ**ダブルクリック**あるいは右クリックのメニューから “ブロックパラメータ” を選択すれば、そこに、パラメータ設定用のウインドウ (ダイアログボックス) が表示されるので数値を代入する部分でキーボードから数値を入力する。図??左は Integrator, 右は Constant のデフォルトでのダイアログボックスである。全てのブロックのダイアログボックスには “Help” ボタンがあり、これをクリックすれば、その

ブロックに関するオンラインマニュアルを参照する事ができる。XY Graph については結果の表示の所で解説する。

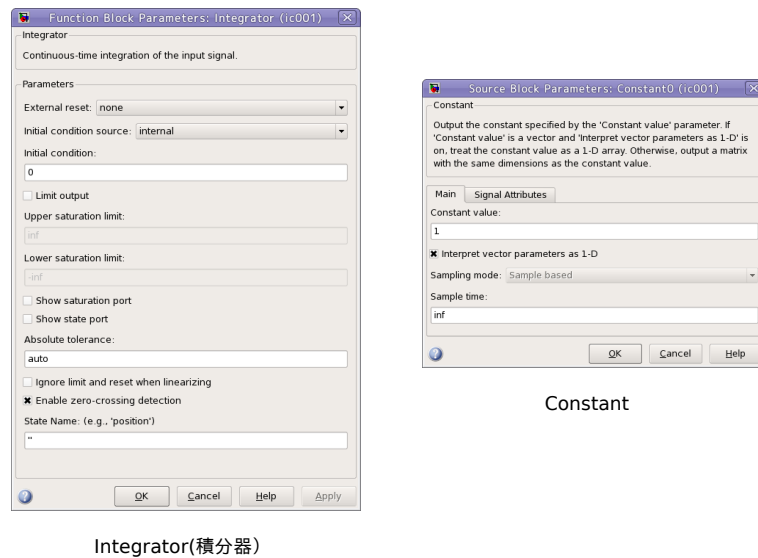


図 20: ブロックパラメータ設定用ダイアログボックス

【Constant ブロックのパラメータ設定】

Constant ブロックでは設定パラメータは“定数値”のみである。Constant0 ブロックをダブルクリックし、ダイアログボックスを表示させる。“定数値”値設定部分にデフォルト値:1 が白黒反転表示されているからそこにマウスカーソルを移動させ一回クリックすると、値をキーボードから入力できるようになるので、デフォルト値を Del キー等で消し、代わりにこの例で用いる値:4.0 を入力する。ここでリターンキーを押すと値が確定されダイアログボックスは閉じられる。多くの設定項目のあるブロックでは全ての項目の入力が終わるまでリターンキーを押さない事。また、確定のためにはダイアログボックス最下段の OK ボタンをクリックしても良い。Cancel ボタンをクリックすれば設定値は無効となり、ダイアログボックスは閉じられる。同様に Constant1 ブロックの“定数値”を-9.8 に設定する。Constant ブロックでは値が変更されると、モデルウインドウ中に表示されているブロックの中の数値が設定した値に変更されて表示される。

【積分器ブロックのパラメータ設定】

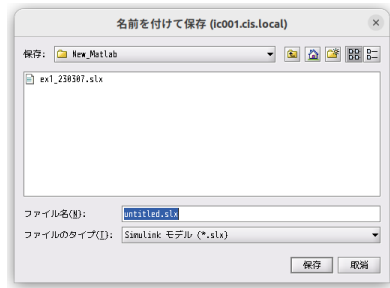
積分器には多くの設定項目がある。最初の二つは初期値 (積分定数) を別の信号で変化させるためのもので設定はサブメニューの選択方式となっている。ここではデフォルトのまま (none,internal) にしておく。次の“初期条件”の数値入力部に初期値を入れる。Constant ブロックと同様にここにマウスカーソルを移動させクリックした後、数値をキーボードから入力する。Integrator0 には状態変数 $x(t)$ の初期値:0.0 を入力する。また Integrator1 には 12.0, Integrator2 には 0.0 を入力する。それ以外の項目はデフォルトのまま変更しない。

1.3 モデルの保存・呼出

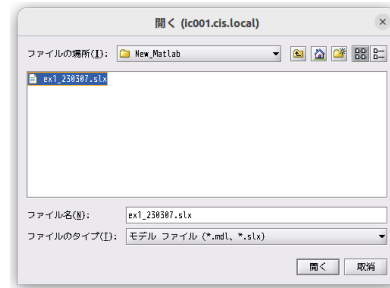
前節までの操作が終ればシミュレーション実行が可能であるが、記述に誤りがある可能性もある。誤りのあるままシミュレーションを実行すると、最悪の場合、システムクラッシュを起こし、記述したモデルが失われてしまう。モデル記述後、あるいは記述途中でも、保存をこまめに行なう事が望ましい。

モデルウインドウの左上のファイルメニューの中から“保存”あるいは“名前を付けて保存”を選択すると、図??左に示すようなファイル指定用ダイアログボックスが表示されるので、ファイル名とパスを指定し適当な場所に保存する。モデルファイルとして以前のバージョンのモデルを呼び出す場合の拡張子は.mdl であるが、現時点で最新のモデルファイル拡張子は.slx である。後者がデフォルトである。

保存されているモデルを呼び出すには、ブロックライブラリウインドウ、あるいは既に開かれているモデルウインドウのファイルメニューから“開く”を選択すれば保存の場合と同様のダイアログボックス (図??右) が開かれるので呼び出したいファイルを指定すれば良い。



モデルファイルの保存



モデルファイルを開く(呼び出す)

図 21: モデルの保存・呼び出しダイアログボックス

1.4 モデルの印刷

モデルウインドウのファイルメニューから“印刷”を選び、さらにサブメニューの“印刷”を選択すれば図??に示したダイアログボックスが開かれる。

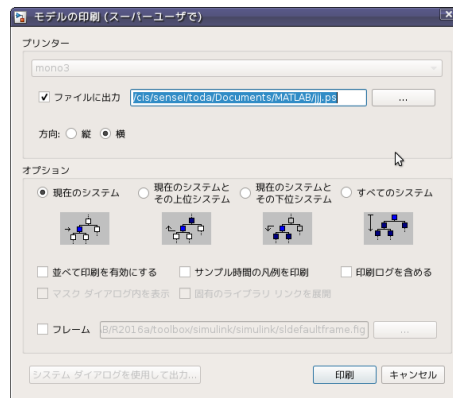


図 22: “モデルの印刷”ダイアログボックス

ドキュメントに貼りこむために、“ファイルに出力”を選び、パス名とファイル名を入力する。デフォルトでは、ファイル名が*.pdfとなっているが、拡張子(.pdf)の部分を書き換えればそうした形式のファイルを出力できる。方向には“横”，オプションでは“現在のシステム”のラジオボタンを選ぶ。フレームは選ばなくて良い。右下の“印刷”ボタンでファイルを格納できる。

しかし、現在のバージョンでは、eps形式で出力しているに関わらず、描画範囲が大きく、.psや.pdfと同じように、無駄な余白が多くなってしまふ。そこでpsやeps, pdfで出力したファイルをinkscapeなどのアプリケーションに一旦読み込み、余白を切り取ってepsに出力すれば良い。

あるいは、画面のキャプチャ (Windows では snipping tool) の方が効率良く品質の良い図が得られる。

2 シミュレーションの実行・モデルの改良

2.1 シミュレーションパラメータの設定

シミュレーションを実行するためには、計算に必要なパラメータを設定する必要がある。モデルウインドウの右下に“auto(ode45)”と表示されている部分を左クリックすると、図??(a)に示したように“ソルバー情報”と書かれたウインドウが開かれるが、その下部右の歯車の記号をクリックすると、同図(b)のコンフィギュレーションパラメータウインドウが開かれる。

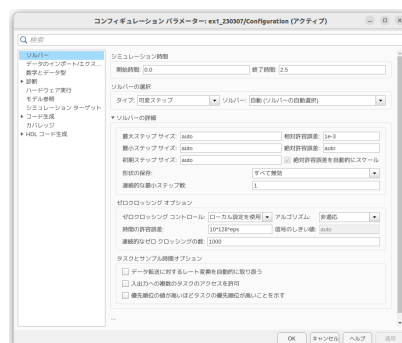
ここでは、まず一番上の“シミュレーション時間”を設定する。開始時間はデフォルトで0.0(秒)が設定されているので、そのままにしておき、この問題の場合は終了時間を2.5(秒)と設定する(この終了時間はモデルウインドウの上部からも設定できる)。

その下の“ソルバーオプション”は微分方程式の数値解法のオプションである。MATLABでは多くの数値解法が利用できるが、ここでは、デフォルトである、“可変ステップ”と“自動(ソルバーの自動選択)”のままにしておく。

また“ソルバーの詳細”の中の“相対許容誤差”は可変ステップ(適応刻み)のステップ幅を決める際のパラメータである⁵。この“相対許容誤差”は結果の信頼性に大きく関与し、問題によってはこの値を非常に小さくしておかなければならない場合がある。この例の場合にはデフォルト値の1.0E-3程度で良いが、後の人工衛星の場合にはさらに小さく(3.0E-14程度)しておく必要があるので注意を要する。その他はデフォルトのままが良い。デフォルトでの解法は可変ステップ幅の公式(Darmond-Prince: 4,5 次の公式(ode45))が選ばれている。その他の項目は変更しない。



(a)モデルウインドウ右下をクリック



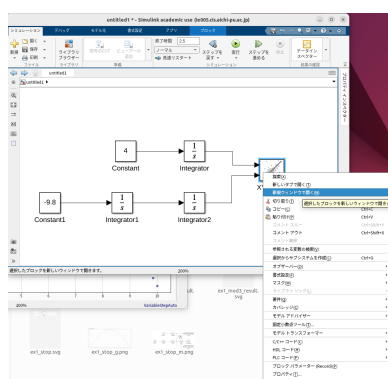
(b)コンフィギュレーションパラメータウインドウ

図 23: シミュレーションパラメータ設定用ダイアログボックスの表示

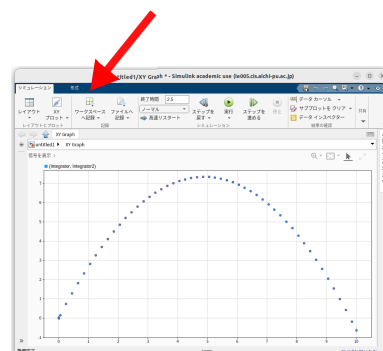
2.2 シミュレーションの開始と終了

シミュレーションパラメータを設定したら、シミュレーションを実行する。モデルウインドウの上部メニューバーの中の実行ボタン“▶”をクリック、あるいはそのすぐ下にある“実行”を選択すると、計算が開始され、モデルウインドウ左下に“準備完了”と表示されれば、計算が終了した事を表している。この例では計算はすぐに終了する。複雑なモデルは長時間にわたる計算が必要な場合もある⁶。

計算が終了したら、図??(a)に示したように“XY Graph”の右クリックで表示されるダイアログの中から“新規ウインドウで開く”を選択すると同図(b)のように別ウインドウが現れ、グラフが表示される。しかし、計算点が点(マーカー)で表示されるのみであるため、これらを線(ライン)で繋ぎ、マーカーを消したい。



(a) 実行結果をxyグラフで確認する
(ダイアログの表示)



(b) 別ウインドウに表示されたグラフ
(表示形式を変更するためのタブ(赤矢印))

図 24: 実行結果の表示

そのためには、グラフウインドウの上タブの中に同図(b)の赤矢印で示したように「形式」タブがあるのでクリックすると、図??(a)に示したようにグラフ形式を変更するためのタブが表示される。赤矢印で示した「ライ

⁵詳細はシミュレーション数理で解説

⁶シミュレーションパラメータである終了時間は、モデルの中での時間であり、計算に必要な時間ではない。僅か0.1秒間の現象のシミュレーションを行なう為にスーパーコンピュータを何日も走らせなければならない計算はいくらでもある。念のため。

ン」タブをクリックすると、同図のように各マーカー間が線で繋がれる。さらに同図 (b) のように「マーカー」タブをクリックし、現れたダイアログの「マーカー」と書かれた」所をクリックすると、図に示されるようにマーカーが消える。もう一度クリックすると再表示される。グラフの横軸と縦軸のスケールが合っていない場合は同図に示したようにグラフ表示領域の右上メニューの中の“ビューに合わせる”(四角のシンボル) を選択すれば自動的に範囲が設定されて表示される。グラフはウィンドウのサイズを変えれば任意の大きさに拡大・縮小できる。

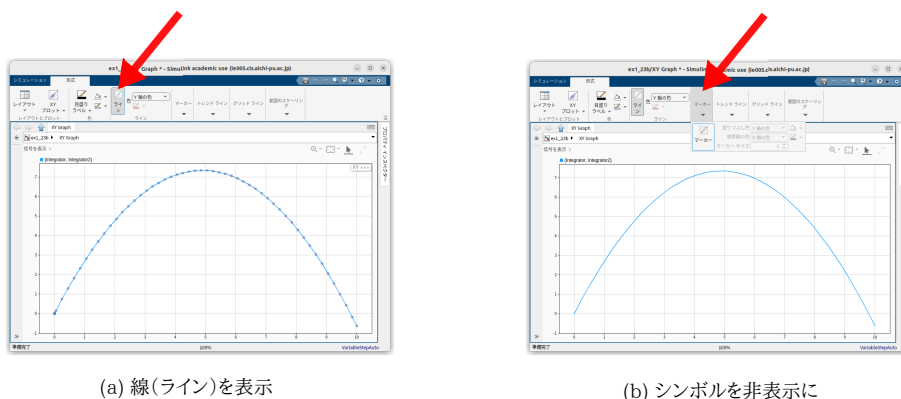


図 25: グラフ形式の変更

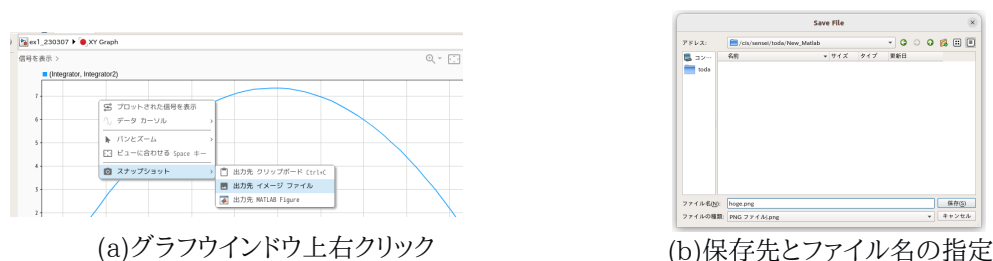


図 26: グラフの保存

また、グラフウィンドウの上部メニューに“ワークスペースへ記録”から Matlab の変数に格納する事もできるので、Matlab の機能を用いてグラフの形式を細かく指定して表示させる事もできる。その場合は Matlab のマニュアルをよく参照して利用する事。さらに“ファイルへ記録”メニューから OS で管理するファイルに保存する事もできるので、他のアプリケーション、または自分で作ったプログラムでデータを利用する事も可能である。

ブロック線図のウィンドウに戻るには、グラフウィンドウの左上の上向き矢印“↑” ボタンを押せば良い。

2.3 モデルの改良

シミュレーションの目的は様々である。微分方程式 (??), (??) でモデル化される対象が“ボールを投げた時のボールの運動”であるとしよう。その際、ある初速度で投げた時の軌道を求める事が目的ならば、前節までのシミュレーションで良い。ところが、いま“どの角度で投げた時最も遠くに到達するか”という問題を解く事となったとき、未だ不十分である。ここでは次の観点からモデルを段階的に改良していく。

- 1 $(x(t), y(t))$ が $t > 0$ で再び $y(t) = 0$ となる t でも $x(t)$ の値 (物体: ボールの到達距離) を求める。
- 2 速度 $(\sqrt{v_x^2(0) + v_y^2(0)})$ を一定とし、角度を入力と見てこれを順次変えながら到達距離を求める。

2.3.1 状態変数の値による条件終了

シミュレーション実行中にモデルの状態 (変数) は変化していくが、ある状態になったらシミュレーションを終了させたい場合のために Simulink では“Stop”ブロックが準備されている。ブロックライブラリ:Sinks(図??左下)の中の最下段に 8 角形の Stop ブロック (図??左) が納められている。これをダブルクリックすると、図

??右のようなパラメータ設定ダイアログボックスが開かれるが、これには設定すべき項目は無い。Stop ブロックには“0 でない値が入力された場合、シミュレーションを終了する”という属性が与えられている。



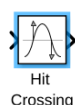
Stopブロックダイアログボックス

図 27: Stop ブロックとそのパラメータダイアログボックス

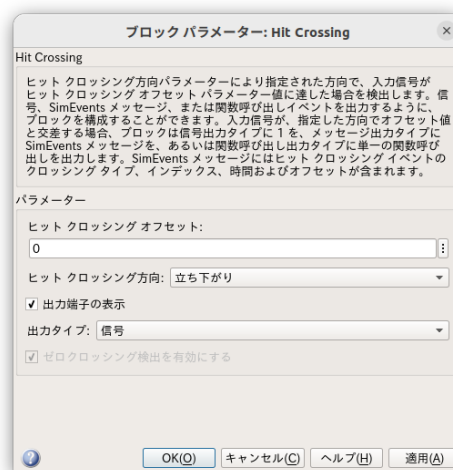
これを利用するためには連続的に変化する状態変数の値を加工して Stop ブロックに入力する必要がある。

Simulink では入力値がある設定値を越えると、高さが 1 の短いパルス (1 計算ステップ幅) を発生する “Hit Crossing” ブロック (図??左) がブロックライブラリ: Discontinuous に納められているのでこれを利用する。これをモデルウィンドウ内にドラッグすると、ヒットクロッシングオフセットの値を入力するボックスが現れる。ので、ここでは 0 を入力する。

“ヒットクロッシング方向” は設定した基準値 (オフセット) と入力値がどのように交叉した場合に検出 (出力が出る) するのかを設定する。値が大きい方から小さい方に変化している場合を検出したいならば図のように“立ち下がり”を選択する。



Hit Crossingブロック

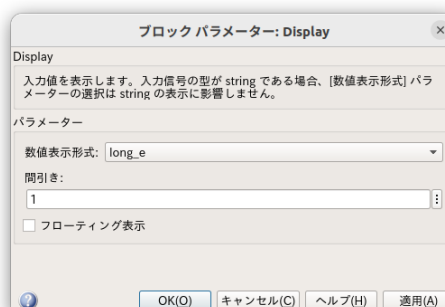


Hit Crossingブロックダイアログボックス

図 28: Hit Crossing ブロックとそのパラメータダイアログボックス



Displayブロック



Displayブロックダイアログボックス

図 29: Display ブロックとそのパラメータダイアログボックス

また、到達点の x 座標を正確に知るために、Sinks ブロックライブラリに納められている “Display” ブロック (図??左) を用いる。これは入力される値を数値表示する機能を持つ。“Format” の選択肢で表示する数値の桁を多くする事が出来るので、“long_e” 等を選んでおくのが良い。桁数を多くした場合、表示の幅が足りない。一度 Display ブロックを選択しておき、四隅の“□”をドラッグし、横に引き伸ばして全桁表示できるようにする必要がある。

これら3つのブロックを先に構成したブロック線図??に接続する。ブロック線図を呼び出して Hit Crossing ブロック, Stop ブロック, Display ブロックをモデルウインドウに置く。

シミュレーションの終了条件は $y(t)$ の値に依るから, Integrator2 の出力を分岐させて Hit Crossing ブロックに接続する。ここで接続線を分岐させる方法について説明する。接続線を分岐させるには, 図??左に示したように接続線上分岐させたい位置で **Ctrl キー**を押しながらドラッグする。ボタンを離すと矢印付近に3方向に青色の矢印が表示されるので, 進みたい方向の上にマウスを合わせると, 同図中央に示したように白抜きの矢印が現れ, 他の青色矢印は消える。さらにこの白抜き矢印を適当な位置までドラッグし, ボタンを離すと接続線が引かれる。

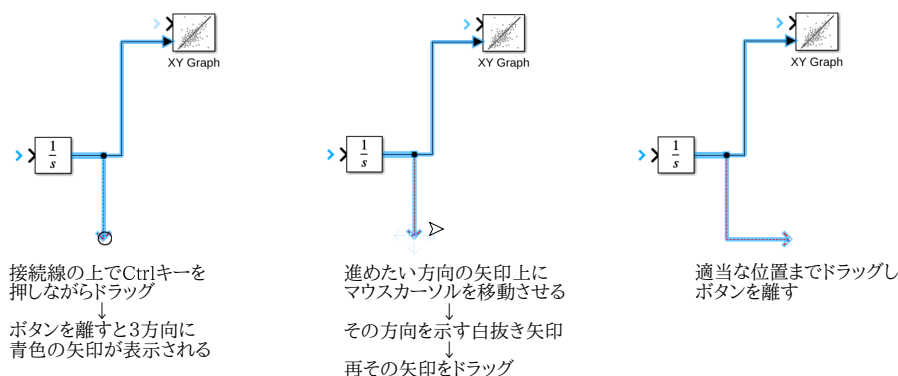


図 30: 接続線の分岐

同様に Integrator0 の出力を分岐させ Display ブロックに接続し, Hit Crossing ブロックの出力は Stop ブロックに接続する。また, $y(t)$ の値が 0 となる時に終了しているかどうかを確認するために Integrator2 の出力をさらに分岐させてもう一つの Display ブロックに接続する。

次にブロックのパラメータを設定する。Stop ブロックでは先に説明したようにパラメータはない。Display ブロックもデフォルトのまま特に設定する必要はない。Hit Crossing ブロックで検出したい設定値は 0 であるため, “オフセット” の値は 0 でよい。また, 先のシミュレーション結果: 図??では, 軌跡 $(x(t), y(t))$ が x 軸と交わるのは 2ヶ所あり, 1つは, シミュレーション開始時点の原点, もう一つが検出したい到達点である。0 と交差するこれら 2つの点の内, 原点では負から正の方向に上昇する状態で, 到達点では正から負の方向に降下する状態で交差する違いがある。ここでは地面に上方から到達する事を検出したいので, “立ち下がり”を選ぶ。このようにして図?? 左に示した新たなモデルを構成する。

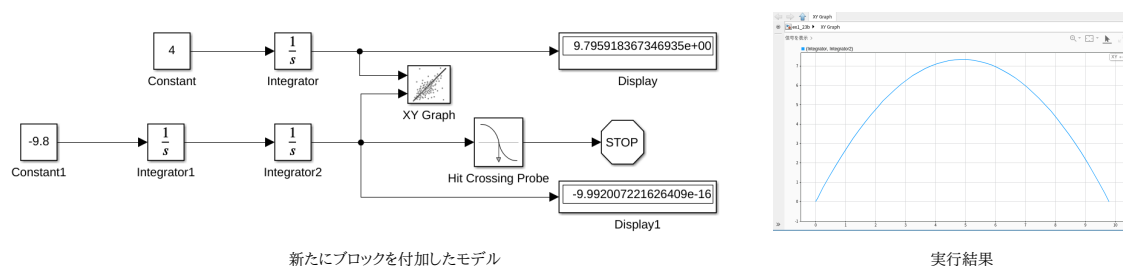


図 31: 新たにブロックを付加したブロック線図 (左) とその実行結果 (右)

新たなブロック線図には名前を付けて保存し (ファイルメニューから “名前を付けて保存” を選択), シミュレーションを実行すると, 図??右に示した結果が得られる。図?? では, 0 レベル越えて負の側に入り込んでいた (投げたボールが地中深く潜っていく) が, グラフ上 0 レベルで止まっていることが判る。また, Integrator1 に接続した Display ブロックにはほぼ 0 に近い値が表示されており, Hit Crossing ブロックと Stop ブロックが正しく働いている事が判る。この時, Integrator0 に接続された Display ブロックに表示されている数値が求めたい到達距離である。

2.3.2 積分器初期値の外部設定

物体を放出する時の速度 ($= |\mathbf{v}| = \sqrt{v_x^2(0) + v_y^2(0)}$) を一定とし，放出角 $\theta(^{\circ})$ を順次変化させる場合， $v_x(0)$ および， $v_y(0)$ を θ を用いて表すと，

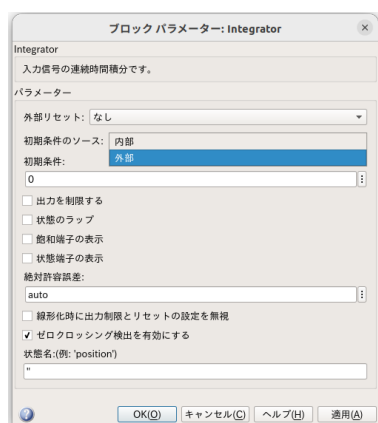
$$\begin{cases} v_x(0) = |\mathbf{v}| \cos\left(\frac{\pi}{180}\theta\right) \\ v_y(0) = |\mathbf{v}| \sin\left(\frac{\pi}{180}\theta\right) \end{cases} \quad (2.1)$$

となる．特に $v_y(0)$ は図??のブロック線図中，積分器: Integrator1 の初期値であり角度を変化させる度に式 (??) を電卓等で手動で計算し，初期値を直接設定するのは面倒である．

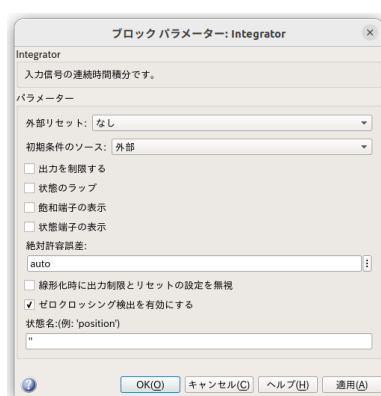
この例の場合のように，モデルの状態変数 (積分器出力) の初期値を種々の条件で変化させシミュレーションを行ないたい場合が頻繁にある．積分器のブロックパラメータには初期値を外部からの入力値に設定できるオプションが付いている．ここでは積分器の初期値の外部設定，三角関数 (Trigonometric Function) ブロック，定数倍 (Gain) ブロックの説明を行う．

【積分器初期値の外部設定】

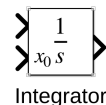
積分器ブロックのダイアログボックス (図??中央) 中の“初期条件のソース”がデフォルトでは“内部”が選択されているのでこれをクリックすると，図??左に示したサブメニューが開かれるのでその中から“外部”を選択する．初期条件欄が無くなる (同図中央) ので，ここで OK をクリックする．積分器ブロックは入力が2つになっている．上側が被積分入力で，下側が初期値を入力するポートである．



ダイアログボックスで“外部”を選択



“外部”選択後のダイアログボックス
(初期値入力欄が無くなっている)

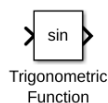


積分器ブロックの入力が2つになっている
(上:被積分入力,下:初期値入力)

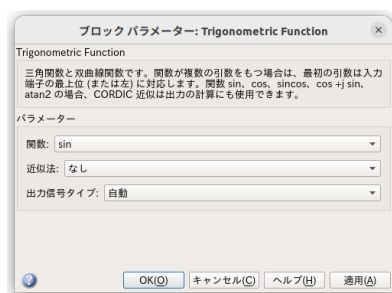
図 32: 積分器初期値の外部設定への変更

【三角関数ブロック】

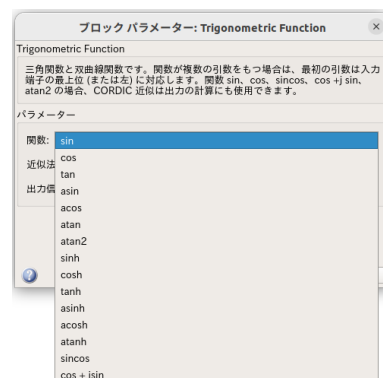
sin ブロックと cos ブロックは算術演算 (Math operators) ブロックライブラリの中の “Trigonometric Functions” をモデルウインドウにドラッグし，ダブルクリックすると，“関数”を設定できるサブメニューが現れるので，その中から “sin” および，“cos” を選べば良い．“Sin Wave Function”とは違うので注意．



三角関数ブロック



三角関数ブロックのダイアログボックス



三角関数を選択する

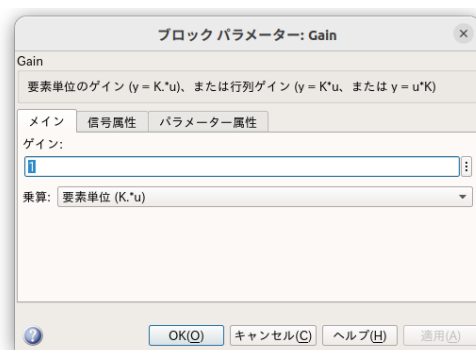
図 33: 三角関数ブロックとそのダイアログボックス

【定数倍ブロック】

定数倍 (Gain) ブロックも算術演算ブロックライブラリに納められている。モデルウィンドウに Gain ブロックを置くと、この場合も“ゲイン”の入力を促すボックスが現れる。図??に定数倍ブロックとそのダイアログボックスを示す。ダイアログボックス中、gain を入力する欄があるのでそこにキーボードから数値を入れる。数値代入後、ブロック中の数字は“K”となる場合があるが、これは表示桁数が足りないため何らかの数値が代入されている事を示すものである。数値を見えるようにしたい場合はブロックを選択しそれをマウスで左右に広げれば数値が現れるが、場所を取るためどうしても必要な所だけにした方が良い。



定数倍ブロック



定数倍ブロックのダイアログボックス

図 34: 定数倍ブロックとそのダイアログボックス

これらを用いて式 (??) に相当する部分を構成し、図??のブロック線図に変更を加えたものが図??である。ここで、Constant ブロックの値には角度: θ ($^{\circ}$) の値を設定し、その直後の定数倍ブロックには、 $0.017453293 \left(= \frac{\pi}{180} \right)$ が設定されている (桁数が多いため、ブロックには初めの 0.0 の部分しか表示されていない。 “pi/180” と入力しても良く、この方が正確な値となる)。sin, cos ブロックの直後の定数倍ブロックには、放出速度 ($|v| = 10.0\text{m/sec}$) が設定されている。

図??のモデル左端の Constant ブロックの値を順次変えながらシミュレーションを行ない、Integrator0 の出力に接続されている Display ブロックの表示を記録し、Matlab のグラフあるいは GnuPlot などを用いれば角度を横軸、到達距離を縦軸としたグラフを作成できる。尚、角度を自動的に変更して行く方法に関しては、参考 1 を参照の事。

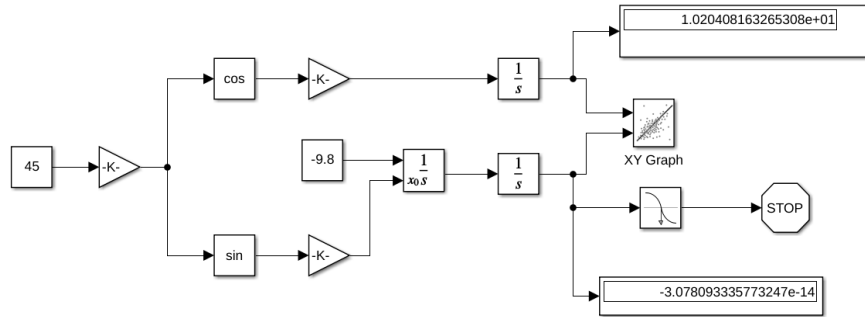


図 35: 放出角度設定部分を付加したブロック線図

3 非線形への発展

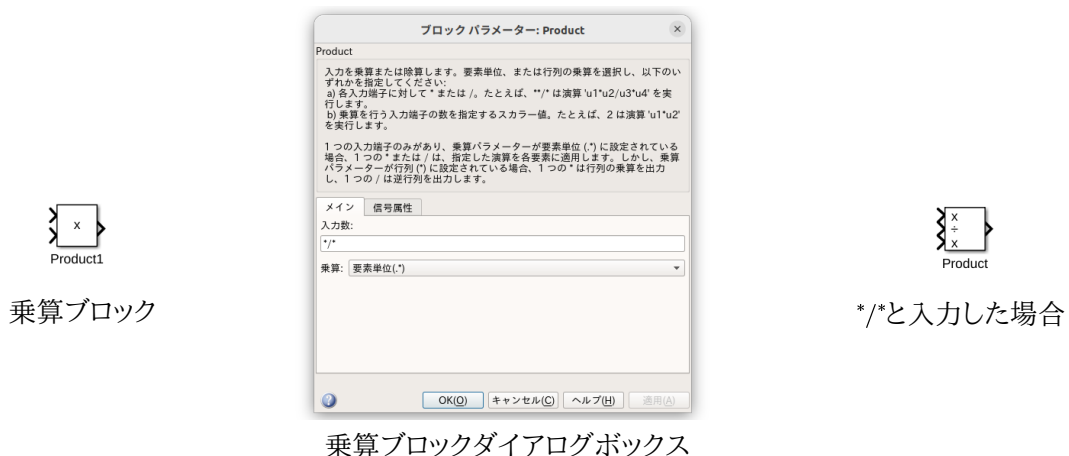
地球上で放物運動を考える際、空気抵抗の考慮が必要である。状態方程式は

$$\frac{d}{dt} \begin{pmatrix} x(t) \\ v_x(t) \\ y(t) \\ v_y(t) \end{pmatrix} = \begin{pmatrix} v_x(t) \\ -\alpha v_x(t) \sqrt{v_x^2(t) + v_y^2(t)} \\ v_y(t) \\ -g - \alpha v_y(t) \sqrt{v_x^2(t) + v_y^2(t)} \end{pmatrix} \quad (3.1)$$

となる。右辺に状態変数間の積が現われるため非線形である。積は算術演算ブロックライブラリに納められている“Product”（乗算）ブロックにより、計算できる。また、2乗も乗算ブロックの2入力を同じにすれば計算できるが、乗算と同じく算術演算ブロックライブラリに納められている“Math Functions”ブロックを用いても良い。また、ここでは用いない（後出する）が良く使う演算として除算があり、これは乗算ブロックを用いて行なう事が出来る。ここではこれらの他加算ブロックについても説明する。

【乗算ブロック】

乗算ブロックは図??左に示したようにデフォルトでは2つの入力を持っているが、任意の数の入力とする事が出来る。ブロックをモデルウィンドウに置くと、この入力数を尋ねるボックスが現れる。



乗算ブロックダイアログボックス

図 36: 乗算ブロックとダイアログボックス

このブロックのブロックパラメータのダイアログボックス（図??中央）の“入力数”欄に欲しい入力の数を設定すれば、ブロックにその数だけの入力ポートが表示される。また、乗算ブロックは入力毎に乗算入力か、除算入力かを独立に設定できる。例えば入力1が乗算、入力2が除算、入力3が乗算の場合、ダイアログボックスの“入力数”欄に数値でなく、 */* と設定すれば良い。 * 記号は乗算、 / 記号が除算に対応しており、

入力ポートの数はこの二種類の記号が何個書かれているかに対応している。尚、この設定法に関する詳説はダイアログボックスの Help ボタンをクリックし、オンラインヘルプで見る事ができる。

図??右にこの場合のブロックの表示を示す。中央の入力に (÷) 記号が表示され入力 2 が除算入力である事が判る。

【加算ブロック】

加算 (Sum) ブロックも乗算ブロックと同様に任意の数の入力ポートを置く事が出来る。Sum 以外にも Add, Subtract ブロックが加算機能を持っている。図??に示したダイアログボックス最下段の “List of signs” 欄に + あるいは - の記号列を記入すれば、ブロックにはその記号の数分だけの入力ポートが描かれ、反時計回りにそれぞれのポートに対応する記号が付される。また | 記号はスペースを意味し入力ポートは描かれない。図??右に -+ および, ++| -|++ と設定した場合に表示されるブロックを示す。この設定法に関してもオンラインヘルプで詳細を見る事ができる。

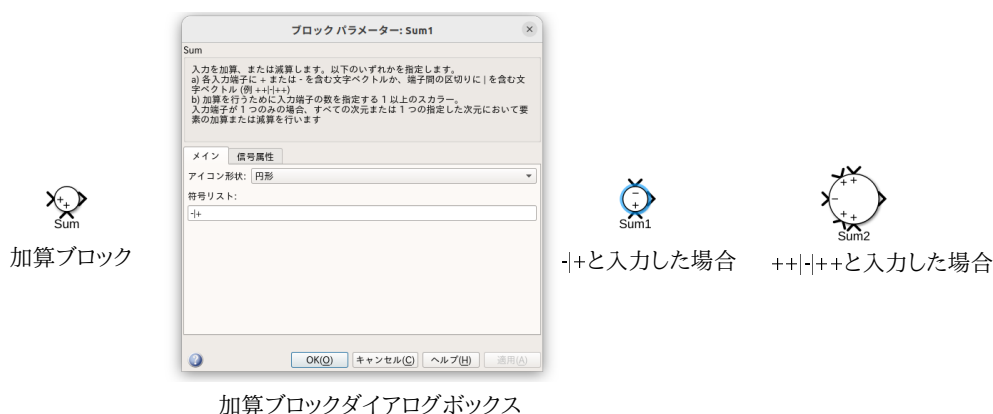


図 37: 加算ブロックとダイアログボックス

【Math Function ブロック】

Math Function ブロック (図??左) では良く知られた幾つかの数学関数の中から一つを選択して用いる事ができる。ダイアログボックス (同図中央) の “Function” の選択項目の中には右に示した幾つかの関数が準備されているので、例えば入力の値の 2 乗を計算するのであれば “square” を選ぶ。ブロックには、選択した関数の名前が描かれる。

また Math Function とは別に平方根関数 (Sqrt) がある (以前のバージョンでは Math Function に含まれていた)。

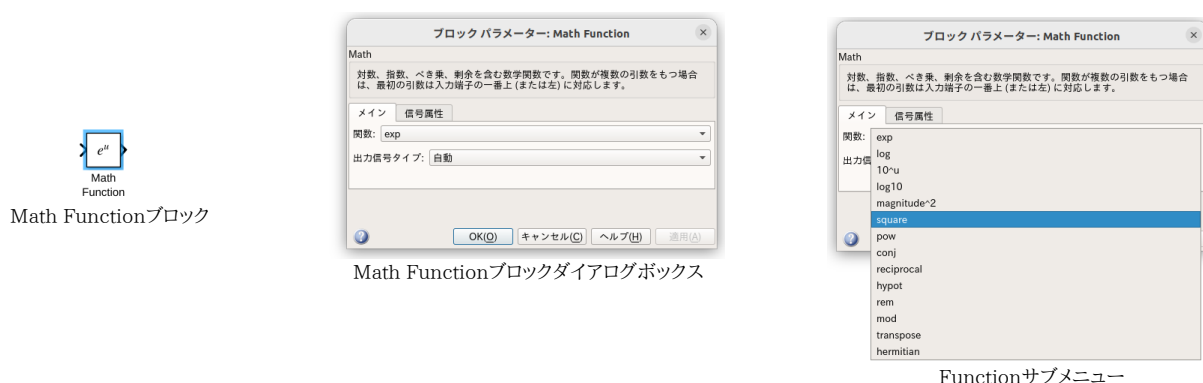


図 38: Math Functions ブロックとダイアログボックス

これらを用いて図??のブロックダイアグラムを元に空気抵抗のある場合 ($\alpha = 0.3$) のブロックダイアグラムを図??のように構成した。また図??は初速度 $10\text{m} / \text{sec}^2$, 放出角度 40 度での実行結果である。空気抵抗によって、到達距離は短くなり、また、軌跡も放物線でなくなっている事が判る。

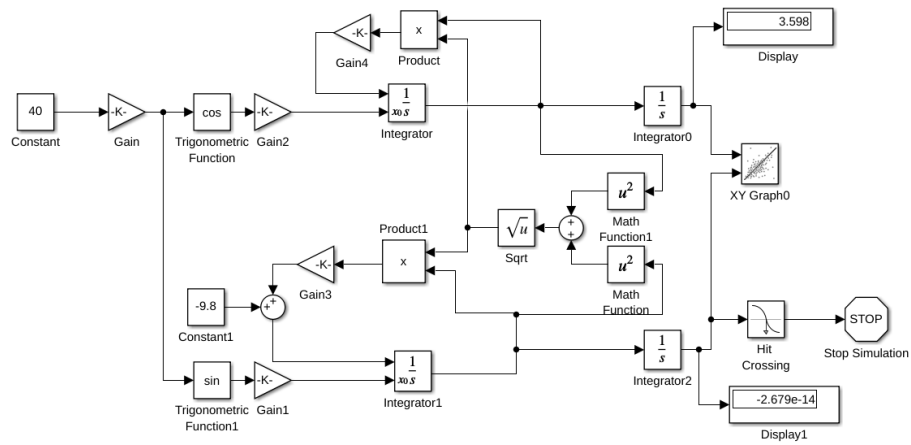


図 39: 空気抵抗のある場合のブロック線図

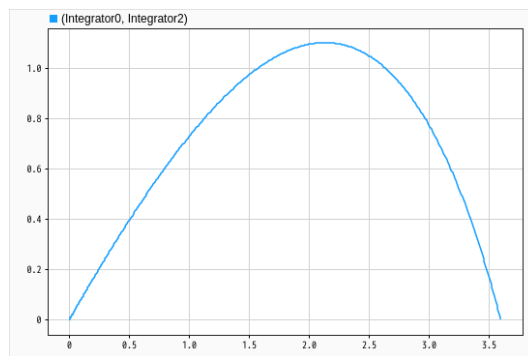


図 40: 空気抵抗のある場合の実行結果

4 参考 1: 初期値設定の自動化

??で積分器初期値を外部から与える方法を示したがさらに複数のシミュレーションを自動化できないだろうか。例えば放出角度を 1 度あるいは 0.5 度といった間隔で変化させて到達距離を求めたい場合は極めて複雑な作業となる。条件を種々変化させながらシミュレーションを実行するためには、種々の方法が考えられるがここではブロック線図で可能なものの一つを構成してみる。勿論他のスマートな方法も考えられるので、ここでは参考とする。

4.1 積分器状態のトリガによるリセット

積分器のダイアログボックスには、積分器出力 (初期値あるいは状態変数の値) を外部のトリガによってリセットできるようになっている。これを使えば任意の時刻でモデルを任意の状態に設定できる。積分器ダイアログボックスの“外部リセット”のサブメニューではトリガの位相 (“立ち上がり”, “立ち下がり”, “両方”) などが選択できる。これを“なし”以外に設定すると、図??に示したように、積分器ブロックに新たな入力が表示され、箱の内部にトリガ入力の位相によって異なる記号 (立ち上がり, 立ち下がりが直観的に理解出来る) が表示される。今回の例では、到達点を保持してから状態をリセットしたいため、Hit Crossing ブロックが発生するパルスの立ち上がりで出力を保持し、パルスの立ち下がりで全ての積分器をリセットする。

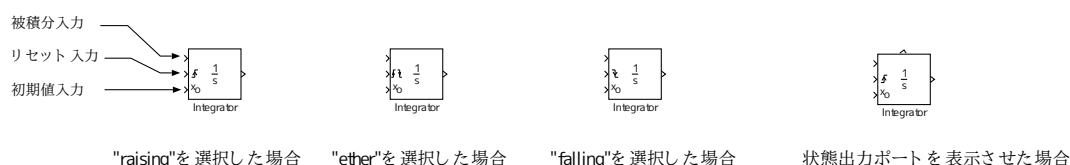


図 41: トリガリセット入力及び、状態出力ポートを持つ積分器

また、3つの積分器のリセットトリガとしては、図??の中の Hit Crossing ブロックの出力を用いる。但しこの時、積分器出力を Hit Crossing ブロックを通してそのまま積分器リセット入力に加えると“代数ループ”と呼ばれる計算不能状態となるので、こうした問題を避けるために積分器ブロックでは、僅かな時間だけ遅らせた出力を出すポートを表示させる事ができる。これを状態出力ポートという。ダイアログボックス中“状態端子の表示”ボタンを押した状態にすれば、図??右端に示したようにブロック上部に状態出力ポートが表示される。リセット入力に接続される出力は、この状態出力ポートに接続しなおす必要がある。

4.2 Sub System によるサンプル&ホールド及び次の状態の計算

さらに、リセット時の $y(t)$ の値を保持する必要があるが、Simulink ではトリガによるサンプル&ホールド素子が無いので、Sub System を用いてこれを構成しよう。ここでは、トリガによって起動する機能を持つ Sub System を用いる。Ports & Subsystems ライブラリの中から“Subsystem”ブロック (図??(左)) あるいは“Atomic Subsystem”をモデルウインドウにドラッグする。これをダブルクリックすると、ダイアログボックスの代わりに別のモデルウインドウが開かれる。即ち“Sub System”とは、利用者が独自に機能を定義できるブロックであり、その機能は Simulink で提供されるブロックで構成されるものである。Sub System はモデルのブロック線図が複雑になる場合や、今回のようにモデル内の状態の変化を検出してある機能を発動させたい場合等に用いられる。ここでは図??に示したような Sub System を構成する。この Sub System の機能は、トリガ入力がある度にその時点での入力ポートの値をホールドし、出力ポート (図中 Out) に出力する。同時に最初に 0 が記憶されている記憶領域の値を呼び出し、1 を加えて出力ポート (図中 initial) に出力すると同時に 1 加えられた値を元の記憶領域に戻す。以下各ブロックについて簡単な説明を行なう。

4.2.1 トリガポートブロック

Sub System モデルウインドウ (図??右) 内最上部に置かれているパルスのシンボルが描かれたブロックをトリガポートブロックという。これがモデルウインドウ内にある Sub System は入力されるトリガによって起動される Sub System である (Triggered sub system という) 事を意味する。トリガポートブロックは Sub System でしか用いる事が出来ない。これが Sub System のモデルウインドウに置かれるとその時点で Sub System ブロックの上部にトリガ入力ポートが描かれる。トリガの位相は積分器のリセットの場合と同様に“立ち上がり”、

“立ち下がり”, “両方” を選択できるここでは Hit Crossing ブロックの出力を用いるので, “立ち上がり” を選ぶ. 尚, Ports & Subsystem ライブラリ内にはトリガーポートブロックを既に含んだサブシステム “Triggered Subsystem” が準備されているので, それを用いても良い.

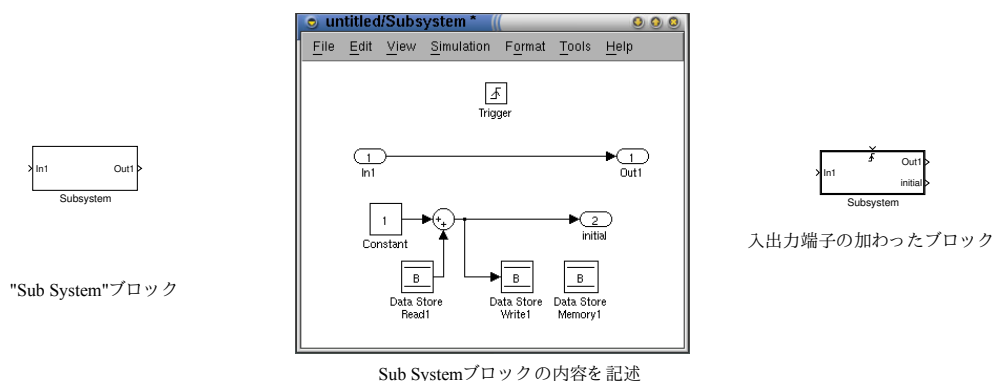


図 42: Subsystem ブロックとその内容

4.2.2 入出力ポートブロック

横長の楕円状の小さいブロックで内部に 1,2 等の数字が描かれたもので, 出力ポートのみが付いているブロックが入力ポートブロック, 入力ポートのみの付いたものが出力ポートブロックである. これらを Sub System モデルウィンドウ内に置くと, 対応する数だけ Sub System ブロックの左右に入出力ポートが描かれる. これは Sub System とこれを呼び出すモデルとの信号のやりとりをするためのものである. 図??では入力ポートブロックから直接出力ポートブロックに接続されていて何もしないように見えるが, この Sub System 全体がトリガによって起動されるため, 次のトリガが来るまでは, 出力ポートには, 前回の入力ポートの値が保持 (ホールド) される. 即ち, この部分はトリガによるサンプル&ホールド素子として機能する.

4.2.3 記憶領域 (Data Store) ブロック

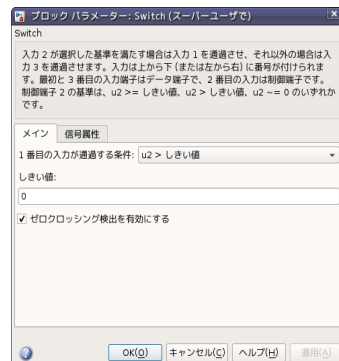
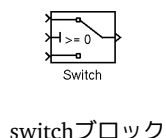
Hit Crossing ブロックによって $y(t) = 0$ が “falling” で検出されるとある角度での放物シミュレーションが終了した事になる. 次に行なう事は放出角度を変化 (1 度加える) させ, 積分器の初期値をこれに対応するように設定し, シミュレーションを続行する事である. Sub System 内ではこの “前回の角度値に 1 を加える” 操作をトリガを合図に行なわせる. そのために, 記憶領域から値を読み出す “Data Store Read” ブロック, 記憶領域に値を書き込む “Data Store Write” ブロック, 及び記憶領域本体 “Data Store Memory” ブロックを用いる. これらは Signal Routing ブロックライブラリに納められている. 図??では定数:1 に “B” と描かれた Data Store Read ブロックの出力を加えて出力ポートに接続されているがこれによって前回の角度値に 1 を加える操作に対応している. またその出力を分岐させ元の記憶領域に戻すために, やはり “B” と描かれた Data Store Write ブロックに接続する事によって次回のための記憶を行なっている. ここで, ブロックに描かれた “B” は記憶領域の名前であり, それぞれのブロックのダイアログボックスで設定できる (デフォルトでは “A”). Data Store Read, Data Store Write ブロックは対応する名前を持つ Data Store Memory ブロックがモデルウィンドウ内に無いとエラーを起こす. また, この例では Data Store Read のダイアログボックス中 “Sample time” の値を -1 としておく必要がある (デフォルトでは 0).

4.3 放出角度と到達距離の 2 次元プロット

Sub System の出力がある角度での到達距離を表しているから, これを Y とし, 放出角度を X として XY Graphs ブロックに入力すれば, 目的のグラフが得られる筈である. 但し, この時, Sub System の到達距離保持中は次の角度のシミュレーションが行なわれている筈だから, 積分器への初期値入力から値を取ってくると, 到達距離との整合が取れない. ここは簡単に前回の角度は 1 度少ないという関係を利用して, 現在シミュレーション中の角度初期値から 1 を引く.

4.4 条件スイッチによるシミュレーションの終了

放出角度が 90 度になったらシミュレーションを終らせたい。方法は種々考えられるが、ここでは、Signal Routing ブロックライブラリに納められている “Switch” ブロックを用いる (図??左)。これは入力 3 個あり、入力 2 (中央) の値がダイアログボックス (図??右) で設定する閾値 (しきいち: Threshold) より大きい等しい場合、入力 1(上) の値が出力され、これ以外の場合は入力 2 の値が出力される。



switchブロックダイアログボックス

図 43: Switch ブロックとそのダイアログボックス

Stop ブロックは 0 以外の値が入力された場合にシミュレーションを終了させるから、Switch ブロックの入力 1 に定数:1 を、入力 2 に定数 0 を接続しておけば、放出角度が閾値を越えた時点でシミュレーションが終る。閾値を 91.0 に設定しておけば、放出角度が 90 度のシミュレーションをし終えた時点で放出角度の更新が行なわれ、91 度となり、その時点で Stop ブロックが働くので、91 度のシミュレーションは行なわれない。

4.5 全体の接続と若干の調整

前節までの変更を図??のブロック線図に加え、変更したものを図??に示す。

シミュレーション時間は 90 回程の放出を行なうので 150sec 程に設定 (Simulation Parameter のダイアログボックス中 Stop Time を 150 に) する。また、グラフが三角波のような波形になってしまう場合があるが、その場合、Constant ブロックのいずれかのブロックパラメータの中 (一番下) の Sample Time を inf から 0.002 以下の数値にセットすれば良い。⁷は “XY Graph1” 到達距離が 10 を越えるため、Y の表示の最大値を 12 程度にし、X の範囲は 0 から 90 (度) に設定する。

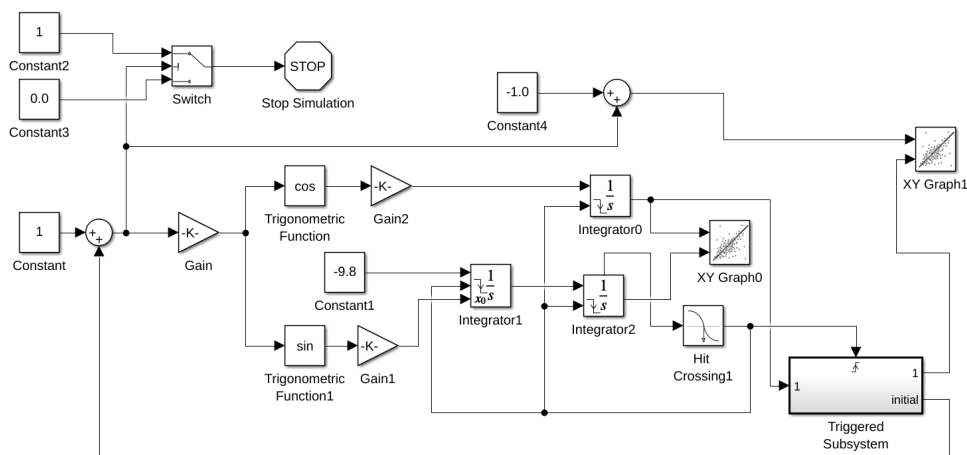
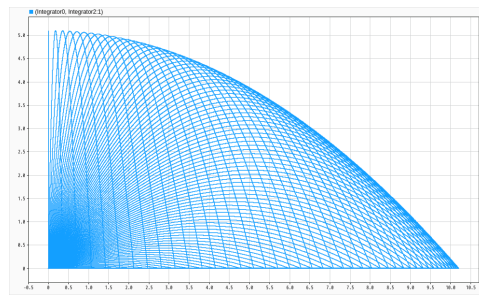


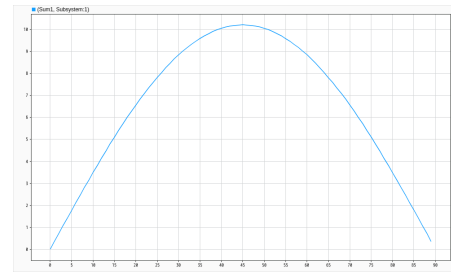
図 44: 初期値の自動設定を加えたブロック線図

⁷2021b より前のバージョンでは XY Graph で Sample Time の設定箇所があったが、新しいものでは無くなっている。

図??に実行結果を示す。



XY Graph0の出力



XY Graph1の出力
(放出角度に対する到達距離)

図 45: 放出角度の変化に対するシミュレーション結果の例

また空気抵抗のある場合のブロック線図とその実行結果は以下になる。

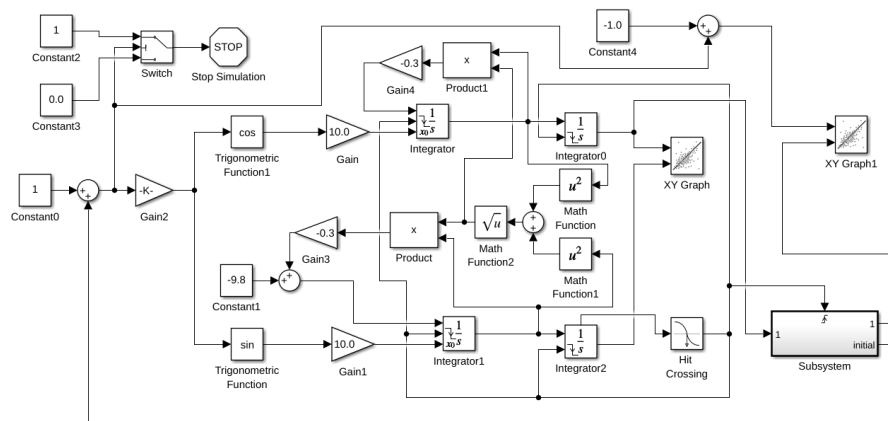
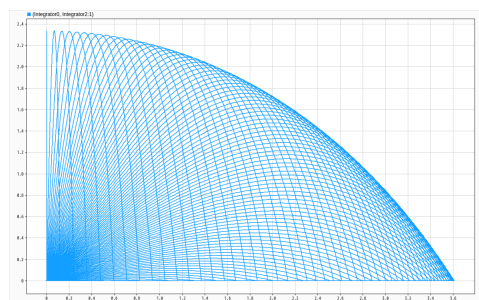
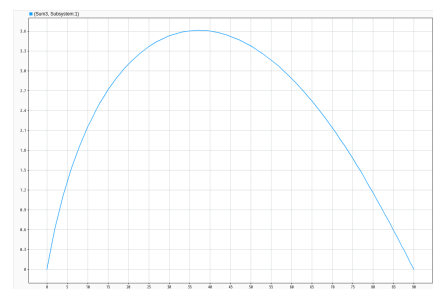


図 46: 空気抵抗のある場合のブロック線図



XY Graph0の出力
(空気抵抗あり)



XY Graph1の出力
(放出角度に対する到達距離)
(空気抵抗あり)

図 47: 空気抵抗のある場合の放出角度の変化に対するシミュレーション結果の例

5 参考 2: To Workspace の利用

本節では、Workspace への保存方法および保存したデータの MATLAB での利用方法について説明する。“Sinks” ライブラリに収められている “To Workspace” を用いると Simulink でのシミュレーションで得られたデータ系列は、Workspace と呼ばれる領域に保存され、MATLAB で利用できるようになる⁸

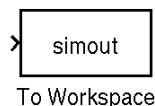
5.1 データ系列の保存

5.1.1 To Workspace ブロックの利用

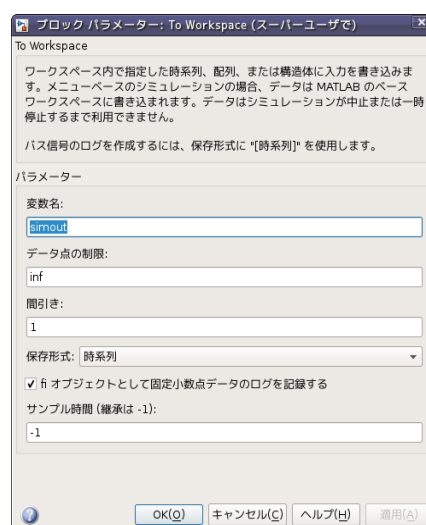
【To Workspace ブロック】

To Workspace ブロック (図 ?? 左) は Sinks ライブラリに分類されている。To Workspace ブロックを用いることで、Simulink で行ったシミュレーションで得られるデータ系列を指定した変数に保存することができる。To Workspace ブロックのダイアログボックスを、図 ?? 右に示す。データ系列は “Variable name” パラメータで設定した名前の変数に “Save format” パラメータで設定したフォーマットにより保存される。同ダイアログボックスに含まれるデータ量・保存タイミングに関するパラメータ, “Limit data points to last”, “Decimation”, “Sample time” については以下のものである。

- “Limit data points to last” パラメータは、保存するデータ数を示す。デフォルトでは `inf` となっており、すべてのデータを保存することを示す。
- “Decimation” パラメータは、間引きファクタである。例えば n を指定した場合、 n 番目のサンプル毎にデータを保存する。デフォルトでは、1 となっており、すべての時間ステップ毎にデータを保存する。
- “Sample time” パラメータは、データ点を収集するサンプリング間隔を示す。デフォルトでは、`-1` が設定されており、接続ブロックからのサンプル時間を継承することを意味する。



To Workspace ブロック



To Workspace ブロックダイアログボックス

図 48: To Workspace ブロックとそのダイアログボックス

“保存形式” パラメータは、“構造体”、“時間付き構造体”、“配列”、“時系列” の 4 種類のフォーマットから選択する。“時間付き構造体” フォーマットは、図 ?? に示す構造をもつ構造体である。なお、入力信号が複数存在する場合、その入力形式によって、`signals` フィールドや `values` フィールドがベクトルとなる (`signals(1)`, `signals(2)` など)。“構造体” フォーマットは、基本的には“時間付き構造体” フィールドに時間データがない (空配列が格納されている) 点で異なる。ここでは“時間付き構造体”あるいは“構造体”を選ぶ。

⁸ “XYGraph” のメニューバーにある “ワークスペースに記録” からでも同様にワークスペースへデータを送る事ができる。

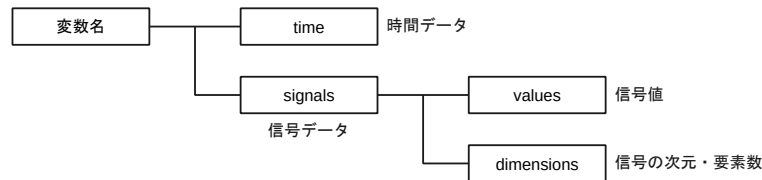


図 49: “Structure with time” のフィールドツリー

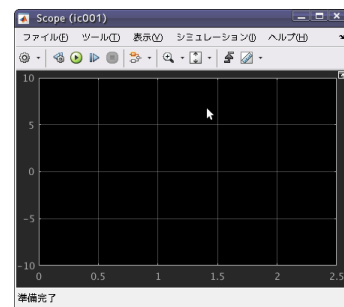
5.1.2 Scope ブロックの利用

【Scope ブロック】

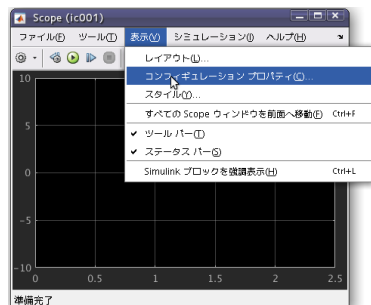
Scope ブロック (図 ?? (a)) は Sinks ライブラリに分類されている。Scope ブロックは、通常、Scope (図 ?? (b)) に入力される信号の時間変化を表示するために用いる。Scope ブロックのコンフィギュレーションプロパティのダイアログ (同図 (d)) は Scope ウィンドウ内のツールバーにある “表示” ボタンをクリックすると現れるサブメニューから起動する (同図 (c))。



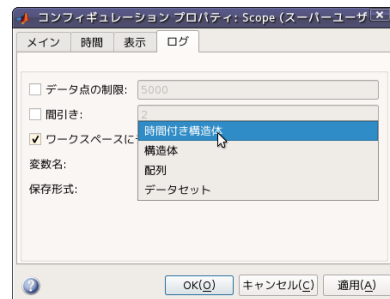
(a) Scopeブロック



(b) Scopeウィンドウ



(c) コンフィギュレーションプロパティの呼び出しは“表示”の中に



(d) コンフィギュレーションプロパティ

図 50: スコープブロックとスコープウィンドウ，コンフィギュレーションプロパティ

Scope コンフィギュレーションプロパティは，“メイン”，“時間”，“表示”，“ログ”の各ページに分かれている。各ページの詳細は省くが，“ログ”ページにおいて，“データ点の制限”パラメータ，ワークスペースへの保存に関するパラメータが設定できる。“ワークスペースにデータのログを作成”のボックスにチェックを入れれば“変数名”と“保存形式”を入力するボックスが有効となる。適当な変数名を入力し，下で説明する保存形式を選ぶ。

5.2 時間データの利用

上述の“保存形式”の中で“時間付き構造体”以外の形式でワークスペースにデータ系列を保存する場合，独立に時間データを得る必要がある。しかし，実は時間データはデフォルトで tout という変数名に保存されている。

モデルウィンドウの“シミュレーション”メニューから ションパラメータ”を選択すると、パラメータ設定用のダイアログボックスが現れるので、“データのインポート／エクスポート”ページを選択する（図 ??）。図 ??をみると、分かるように、時間データは、tout という変数名で“Dataset”（時系列）フォーマットによ

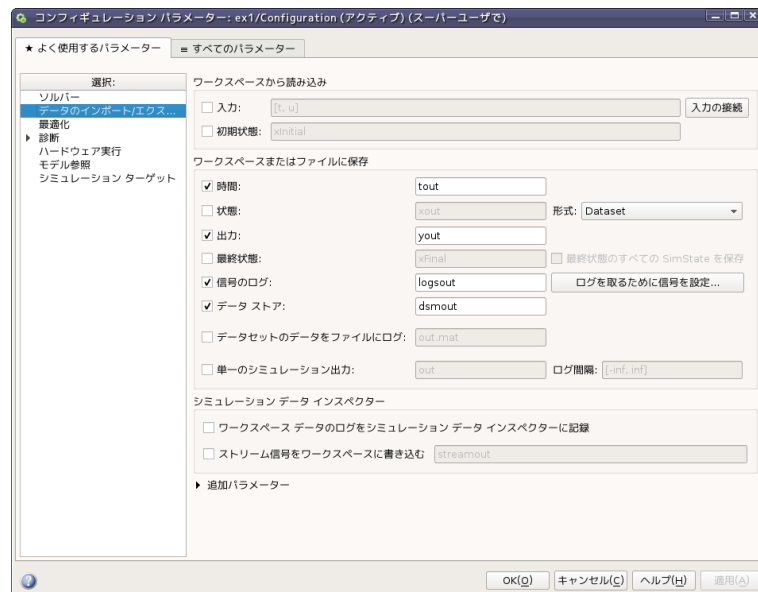


図 51: “データのインポート／エクスポート” 設定ページ

りワークスペースに保存されている。

5.3 例題

図 34 において、 $x(t), y(t)$ のデータ系列をワークスペースに保存し、MATLAB で $y(t)$ の最大値、およびそのときの $x(t)$ の値と時刻 t を求めることを考える。

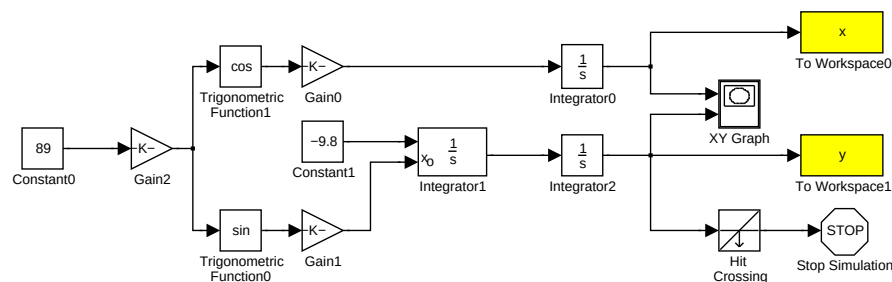


図 52: 図 34 において“Display”ブロックの代わりに“ToWorkspace”ブロックを配置したもの

図 34 において、“Display”ブロックの代わりに、“To Workspace”ブロックを配置する（図 ??）。 “To Workspace0”ブロックは“変数名”パラメータを x, “フォーマット”パラメータを“時間付き構造体”と設定し, “To Workspace1”ブロックは“変数名”パラメータを y, “フォーマット”パラメータを“時間付き構造体”と設定している。

シミュレーション後、MATLAB のコマンドラインで whos コマンドにより変数のリストを表示すると

```
>> whos
  Name      Size      Bytes  Class
  tout      211x1         1688  double array
  x          1x1          4174  struct array
  y          1x1          4174  struct array

Grand total is 1115 elements using 10036 bytes
>>
```

また、変数 x をタイプインすると

```
>> x ↓
x =
    time: [211x1 double]
  signals: [1x1 struct]
 blockName: 'ex13_toWS/To Workspace0'
>>
```

となり、構造体であることを示している。

さて、 $y(t)$ の最大値は MATLAB の組み込み関数 `max()` を用いれば得られる。単に `max(y.signals.values)` とすれば

```
>> max(y.signals.values) ↓
ans =
    5.1005
>>
```

のように最大値が得られるが、

```
>> [ymax,yidx] = max(y.signals.values) ↓
ymax =
    5.1005

yidx =
    107
>>
```

のように実行すれば、`max()` の戻り値を受け取れば、 $y(t)$ の最大値格納されている配列の添字情報が得られる。`yidx` を用いて、 $y(t)$ が最大となるときの、 $x(t)$ の値は

```
>> x.signals.values(yidx) ↓
ans =
    0.1780
>>
```

t の値は `tout(yidx)` または `y.time(yidx)` とタイプインすることで得られる。

```
>> y.time(yidx) ↓
ans =
    1.0200
>>
```